**МИНОБРНАУКИ РОССИИ**
**федеральное государственное бюджетное образовательное учреждение**
**высшего образования**
**«Московский государственный технологический университет «СТАНКИН»**
**(ФГБОУ ВО «МГТУ «СТАНКИН»)**

**Институт**
**информационных технологий**

**Кафедра**
**информационных систем**

**Основная образовательная программа 09.03.02**
**«Информационные системы и технологии»**

# Отчет по дисциплине «Объектно-ориентированное программирование»

**по лабораторным работам № 2–3**

**Тема: «Модель работы вычислительной сети при выполнении задачи»**

**Проверил**
**к.т.н., доцент** **Волков А.И.**

подпись

**Выполнил**
**студент группы <ГРУППА>** **<ФИО>**

подпись

Москва, 2026 г.

# ОГЛАВЛЕНИЕ

ВВЕДЕНИЕ.....3

1 ПОСТАНОВКА ЗАДАЧИ.....4

1.1 Требования к элементам пользовательского интерфейса.....4

1.2 Требования к организации ввода-вывода данных.....6

1.3 Требования к реакции программы на нестандартные ситуации.....6

1.4 Прочие требования.....7

2 ЛОГИЧЕСКАЯ СТРУКТУРА ПРОГРАММЫ.....8

2.1 Дерево наследования узлов обработки.....8

2.2 Дерево наследования сегментов сети.....10

2.3 Классы-наследники узлов.....10

2.4 Вспомогательные классы.....11

2.5 Событийная модель.....11

2.6 Сценарии работы программы.....12

3 СХЕМЫ АЛГОРИТМОВ.....15

3.1 Обработка шага модельного времени узлом.....15

3.2 Ремонт узла.....16

3.3 Маршрутизация пакета сегментом.....17

3.4 Обработка отказа узла серверным сегментом.....18

3.5 Исправление ошибки данных клиентом.....19

3.6 Движение пакета по маршруту.....20

4 ТЕКСТ ПРОГРАММЫ.....22

5 РЕЗУЛЬТАТЫ ТЕСТИРОВАНИЯ.....72

ВЫВОДЫ.....78

# ВВЕДЕНИЕ

Целью работы является разработка приложения «Модель работы вычислительной сети при выполнении задачи» на языке C# с использованием Windows Forms. Приложение должно моделировать прохождение задачи по сети, показывать работу объектов на экране и реагировать на два вида сбоев: ошибку данных и отказ узла.

Работа выполняется по варианту 12 задания «Модель работы вычислительной системы или сети при выполнении некоторой задачи» и включает два задания: разработку классов со свойствами, методами и событиями, а также наследование и событийное взаимодействие объектов.

В первом задании создан основной класс ProcessingNode (узел обработки) с инкапсулированными данными, четырьмя и более свойствами, двумя методами, конструктором и тремя событиями, а также вспомогательные классы Packet, Client, Administrator, EndUser, NetworkSegment, SimulationClock, TrackGraph и другие.

Во втором задании созданы классы-наследники FirewallNode, AppServerNode и DatabaseNode, переопределяющие метод Evaluate, и классы-наследники ClientSegment и ServerSegment, переопределяющие маршрутизацию пакетов. Взаимодействие объектов построено на пользовательских событиях и делегатах.

Для проверки работы в интерфейсе предусмотрены настройки вероятностей сбоев, времени обработки и скорости модельного времени, кнопки «Старт/Стоп», «Задача», «Сброс» и скрытие журнала.

# 1 ПОСТАНОВКА ЗАДАЧИ

Разрабатываемое приложение моделирует выполнение задачи в вычислительной сети, состоящей из двух сегментов. Задача представляет собой пакет, который движется по дорожкам сети и последовательно проходит цепочку узлов:

Коммутатор → Брандмауэр → Сервер приложений → База данных.

При обработке возможны два вида проблем. Ошибка данных возникает на брандмауэре или сервере приложений: пакет-ошибка возвращается клиенту, клиент исправляет данные и отправляет задачу заново с начала цепочки. Отказ узла возникает на базе данных: узел переходит в состояние «Отказ», задачи накапливаются в очереди, администратор подключается к узлу по SSH, выполняет ремонт и отправляет команду Fix; после ремонта обработка продолжается с места отказа, без повторного прохождения цепочки.

При успешной обработке результат возвращается клиенту. Пользователь стоит рядом с клиентом и реагирует на переданный ему результат.

## 1.1 Требования к элементам пользовательского интерфейса

Интерфейс должен быть реализован средствами Windows Forms. Состав элементов управления и их свойства приведены в таблице 1.

Таблица 1 – Элементы управления главного окна

| Имя (Name) | Тип | Свойство Text | Прочие свойства |
|---|---|---|---|
| MainForm | Form | Модель работы вычислительной сети | FormBorderStyle=FixedSingle, MaximizeBox=False, ClientSize=1090×768, StartPosition=CenterScreen |
| pnlSettings | Panel | — | Dock=Top, Height=76 |
| lblDataError | Label | Вероятность ошибки данных: | AutoSize=True |
| cmbDataError | ComboBox | 0,1 | DropDownList; 0,0 / 0,1 / 0,2 / 0,3 |
| lblOutage | Label | Вероятность отказа БД: | AutoSize=True |
| cmbOutage | ComboBox | 0,05 | DropDownList; 0,0 / 0,05 / 0,1 / 0,2 |

| Имя (Name) | Тип | Свойство Text | Прочие свойства |
|---|---|---|---|
| lblProcTime | Label | Время обработки, с: | AutoSize=True |
| cmbProcTime | ComboBox | 2 | DropDownList; 1 / 2 / 3 |
| lblSpeed | Label | Скорость: | AutoSize=True |
| trbSpeed | TrackBar | — | Minimum=1, Maximum=8, Value=2, TickStyle=None |
| chkAuto | CheckBox | Автоотправка задач | Checked=False |
| btnStartStop | Button | Старт | Size=90×27 |
| btnSendTask | Button | Задача | Size=90×27 |
| btnReset | Button | Сброс | Size=90×27 |
| btnLog | Button | Скрыть журнал | Size=130×27 |
| canvas | SimulationCanvas | — | Dock=Fill, DoubleBuffered=True |
| lstLog | ListBox | — | Dock=Bottom, Height=130, HorizontalScrollbar=True |
| statusStrip | StatusStrip | — | Dock=Bottom |
| lblCounters | ToolStripStatusLabel | — | Счётчики модели |
| lblHint | ToolStripStatusLabel | — | Spring=True, TextAlign=MiddleRight |

Общий вид окна приведён на рисунке 1.

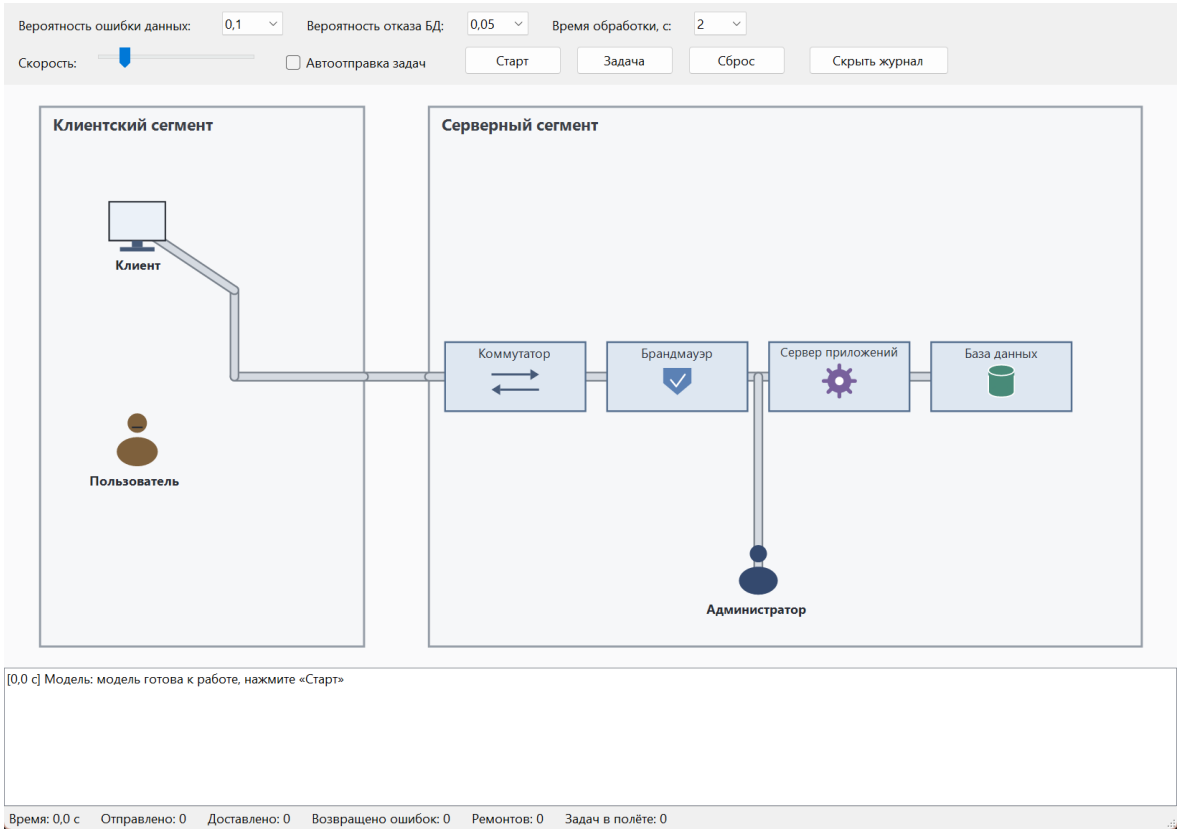



Рисунок 1 – Общий вид главного окна приложения

## 1.2 Требования к организации ввода-вывода данных

Ввод исходных данных должен выполняться пользователем без ручного ввода чисел: вероятность ошибки данных, вероятность отказа базы данных и время обработки выбираются из выпадающих списков, скорость моделирования задаётся движком, автоотправка — флажком. Такой способ исключает ввод недопустимых значений.

Вывод должен выполняться по трём каналам: сцена моделирования (движение пакетов, состояния узлов и сегментов, очередь пакетов), журнал событий с отметкой модельного времени и строка состояния со счётчиками (отправлено, доставлено, возвращено ошибок, ремонтов, задач в полёте).

Работа с файлами в программе не предусматривается: все данные формируются в процессе моделирования, журнал отображается на экране и при необходимости копируется пользователем вручную.

## 1.3 Требования к реакции программы на нестандартные ситуации

Программа должна корректно реагировать на ситуации, перечисленные в таблице 2.

Таблица 2 – Реакция программы на нестандартные ситуации

| Ситуация | Реакция программы |
|---|---|
| Нажатие «Задача» при остановленном моделировании | Задача не создаётся; в строке состояния выводится подсказка «Моделирование остановлено — нажмите «Старт»» |
| Более пяти задач одновременно в работе | Задача не создаётся; выводится подсказка «Достигнут предел числа задач в полёте (5)» |
| Изменение настроек или скорости во время работы | Настройки применяются сразу и действуют на последующую обработку; модель не перезапускается |
| Нажатие «Сброс» при пакетах в пути | Модельное время останавливается, создаётся новый объект SimulationWorld, сцена и журнал очищаются |
| Закрытие главного окна | В обработчике FormClosing модельное время останавливается |
| Недопустимое значение свойства (вероятность вне 0…1, время ≤ 0, скорость вне 0,25…5) | Сеттер свойства бросает ArgumentOutOfRangeException; форма перехватывает исключение и показывает сообщение |
| Пакет без адресата или без маршрута | Пакет отбрасывается, событие PacketDropped пишется в журнал; программа не падает |
| Повторные ошибки данных | Вероятность сбоя уменьшается вдвое за каждую попытку; после MaxAttempts задача отменяется |
| Повторные отказы узла | После ремонта следующая обработка гарантированно успешна, поэтому зацикливание невозможно |

## 1.4 Прочие требования

– модельное время должно быть единственным источником времени; все таймеры объектов работают от события Tick;

– сцена должна масштабироваться под размер окна и корректно отображаться при различном масштабе экрана;

– приложение не должно использовать сторонние библиотеки и внешние файлы изображений;

– все объекты модели должны быть описаны комментариями на русском языке;

– логика модели не должна зависеть от главной формы.

# 2 ЛОГИЧЕСКАЯ СТРУКТУРА ПРОГРАММЫ

Программа построена по схеме «модель — представление». Классы модели находятся в пространстве имён NetworkSim.Model и не знают о форме; главная форма работает с моделью только через класс SimulationWorld, который создаёт объекты и связывает их событиями. Отрисовка сцены выполняется в классе SimulationCanvas (пространство имён NetworkSim.View), который вызывает метод Draw у модели.

Сцена состоит из двух сегментов — клиентского и серверного, — соединённых линией связи через шлюзы. В клиентском сегменте находятся клиент и пользователь, в серверном — четыре узла обработки и администратор. Пакеты движутся по дорожкам, которые описаны графом (класс TrackGraph); маршрут ищется поиском в ширину.

## 2.1 Дерево наследования узлов обработки

Основным классом модели является ProcessingNode. Класс не является абстрактным: его экземпляр создаётся в SimulationWorld и работает как коммутатор. От ProcessingNode наследуются три специализированных узла (рисунок 2).

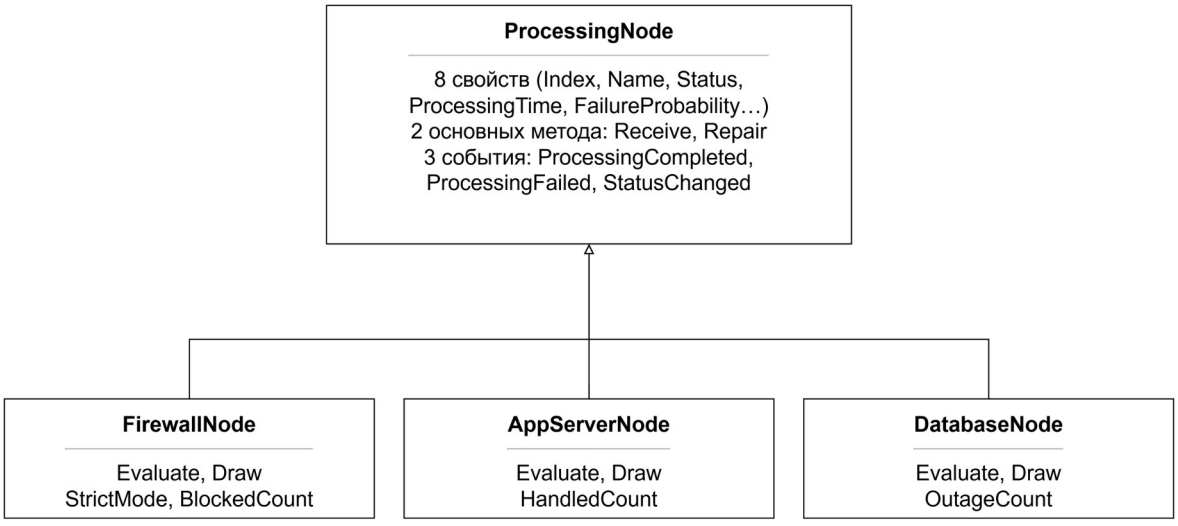



Рисунок 2 – Дерево наследования классов узлов обработки

Класс ProcessingNode содержит восемь свойств (Index, Name, WaypointId, Location, Status, ProcessingTime, FailureProbability, QueueLength), методы Receive, Repair, Evaluate и Draw, конструктор и три события — ProcessingCompleted, ProcessingFailed и StatusChanged. Данные класса закрыты: поля объявлены закрытыми, доступ выполняется через свойства.

Свойство ProcessingTime описано полностью: геттер возвращает значение поля, а сеттер проверяет, что время положительно, и при недопустимом значении бросает ArgumentOutOfRangeException. Свойство Status имеет закрытый сеттер: при изменении состояния оно вызывает событие StatusChanged.

Таблица 3 – Основные члены класса ProcessingNode

| **Член класса** | **Вид** | **Назначение** |
|---|---|---|
| Index | свойство, только чтение | Порядковый номер узла, назначается автоматически |
| Name | свойство, только чтение | Название узла для сцены и журнала |
| WaypointId | свойство, только чтение | Идентификатор вершины графа дорожек |
| Location | свойство, только чтение | Координаты центра узла на схеме |
| Status | свойство, чтение; сеттер закрытый | Состояние: простой / обработка / отказ |
| ProcessingTime | свойство, чтение и запись | Время обработки пакета, с (проверка > 0) |
| FailureProbability | свойство, чтение и запись | Вероятность сбоя, 0…1 (проверка диапазона) |
| QueueLength | свойство, только чтение | Число пакетов в очереди |
| Receive(Packet) | метод | Приём пакета: задача — в очередь, Fix — ремонт |
| Repair() | метод | Ремонт отказавшего узла и продолжение обработки |
| Evaluate(Packet) | защищённый виртуальный метод | Определение исхода обработки; переопределяется |
| EffectiveProbability(Packet) | защищённый метод | Вероятность сбоя с учётом номера попытки |
| Draw(Graphics) | виртуальный метод | Отрисовка узла, очереди и пиктограммы |
| ProcessingCompleted | событие | Пакет успешно обработан |
| ProcessingFailed | событие | Ошибка данных или отказ узла |
| StatusChanged | событие | Изменение состояния узла |

## 2.2 Дерево наследования сегментов сети

Второе дерево наследования образуют сегменты сети (рисунок 3). Базовый класс NetworkSegment содержит граф дорожек, список локальных адресатов и пакетов в пути, методы Submit и Accept, а также события PacketLeaving, PacketDelivered, PacketDropped и StateChanged. Наследники переопределяют метод ResolveDestination, который решает, кому адресован пакет.

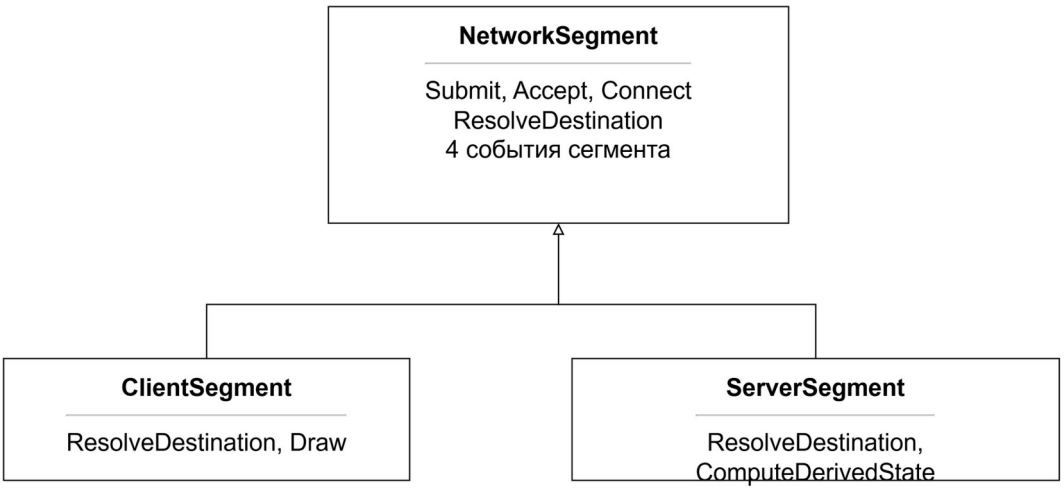



Рисунок 3 – Дерево наследования классов сегментов сети

ClientSegment направляет результаты и ошибки данных клиенту, а задачи — наружу, в серверный сегмент. ServerSegment хранит упорядоченную цепочку узлов: задача направляется первому узлу, уведомление об отказе — администратору, команда ремонта — отказавшему узлу. Если адресат находится в другом сегменте, пакет едет к шлюзу и передаётся соседнему сегменту через событие PacketLeaving.

## 2.3 Классы-наследники узлов

Каждый наследник переопределяет метод Evaluate и метод Draw, добавляя собственную логику сбоя и свою пиктограмму (таблица 4).

Таблица 4 – Классы-наследники узлов обработки

| Класс | Переопределяемые методы | Дополнительные члены | Назначение |
|---|---|---|---|
| FirewallNode | Evaluate, Draw | StrictMode, BlockedCount | Проверка данных; в строгом режиме вероятность ошибки удваивается |
| AppServerNode | Evaluate, OnProcessingCompleted, Draw | HandledCount | Вероятность ошибки растёт с длиной очереди |
| DatabaseNode | Evaluate, Draw | OutageCount | Возможен только полный отказ узла, не зависящий от попытки |

## 2.4 Вспомогательные классы

Класс Packet описывает пакет: тип (Task, DataError, NodeFailure, Fix, Result), номер попытки, владельца задачи, узел-subject для отказа и ремонта, текущую позицию и маршрут. Пакет сам продвигается по маршруту методом Advance и вызывает событие Arrived, когда маршрут пройден.

Класс Client отправляет задачи, принимает ошибки данных и результаты. При ошибке он встаёт в состояние Fixing, выдерживает паузу FixTime и отправляет задачу с номером попытки на единицу больше; после MaxAttempts попыток задача отменяется событием TaskAbandoned. Получив результат, клиент показывает его и передаёт пользователю событием ResultReceived.

Класс Administrator принимает уведомления об отказах, «подключается по SSH» к узлу и через RepairTime отправляет команду ремонта Fix. Класс EndUser стоит рядом с клиентом и реагирует на переданный результат. Класс SimulationClock — единственный источник времени: он вызывает событие Tick с шагом Dt, зависящим от реально прошедшего времени и коэффициента скорости. Класс SimulationLogger подписан на события всех объектов и формирует строки журнала. Класс Layout хранит координаты всех объектов сцены.

## 2.5 Событийная модель

Для событий объявлены два собственных делегата: NodeCompletedHandler и NodeFailedHandler. События ProcessingCompleted и ProcessingFailed объявлены полностью, с явными методами add и remove, а не автоматическими полями. Подписка на них выполнена в другом классе — ServerSegment, в методе SubscribeToNode, через явное создание делегата:

```
node.ProcessingCompleted += new NodeCompletedHandler(OnNodeCompleted);
node.ProcessingFailed += new NodeFailedHandler(OnNodeFailed);
```

Методы OnNodeCompleted и OnNodeFailed являются обработчиками: они решают, что делать с пакетом после завершения обработки или сбоя. Карта основных событий приведена в таблице 5.

Таблица 5 – Карта событий модели

| Издатель | Событие | Подписчик | Реакция |
|---|---|---|---|
| SimulationClock | Tick | сегменты, узлы, акторы, форма | Движение пакетов, таймеры, перерисовка |
| Client | PacketSent | ClientSegment | Построение маршрута и запуск пакета |
| Client | FixStarted | ClientSegment, SimulationWorld | Сегмент переходит в состояние «авария» |
| Client | ResultReceived | ClientSegment, журнал | Передача результата пользователю |
| Packet | Arrived | сегмент, в котором едет пакет | Доставка адресату или передача соседу |
| NetworkSegment | PacketLeaving | соседний сегмент | Продолжение пути в другом сегменте |
| ProcessingNode | ProcessingCompleted | ServerSegment, журнал | Следующий узел или результат |
| ProcessingNode | ProcessingFailed | ServerSegment, журнал | DataError клиенту или NodeFailure |
| ProcessingNode | StatusChanged | ServerSegment, журнал | Пересчёт состояния сегмента |
| Administrator | PacketSent | ServerSegment | Запуск пакета Fix к узлу |
| Administrator | RepairStarted | журнал | Подключение по SSH |
| EndUser | ResultReceived | ClientSegment, SimulationWorld | Состояние «успешно», счётчики |

## 2.6 Сценарии работы программы

Сценарий А (успех). Пользователь нажимает «Старт» и «Задача». Клиент создаёт пакет Task, клиентский сегмент направляет его к шлюзу,

серверный сегмент принимает пакет и передаёт первому узлу. Узлы последовательно обрабатывают задачу, после последнего узла создаётся пакет Result, который возвращается клиенту; клиент передаёт результат пользователю. Результат показан на рисунке 4.

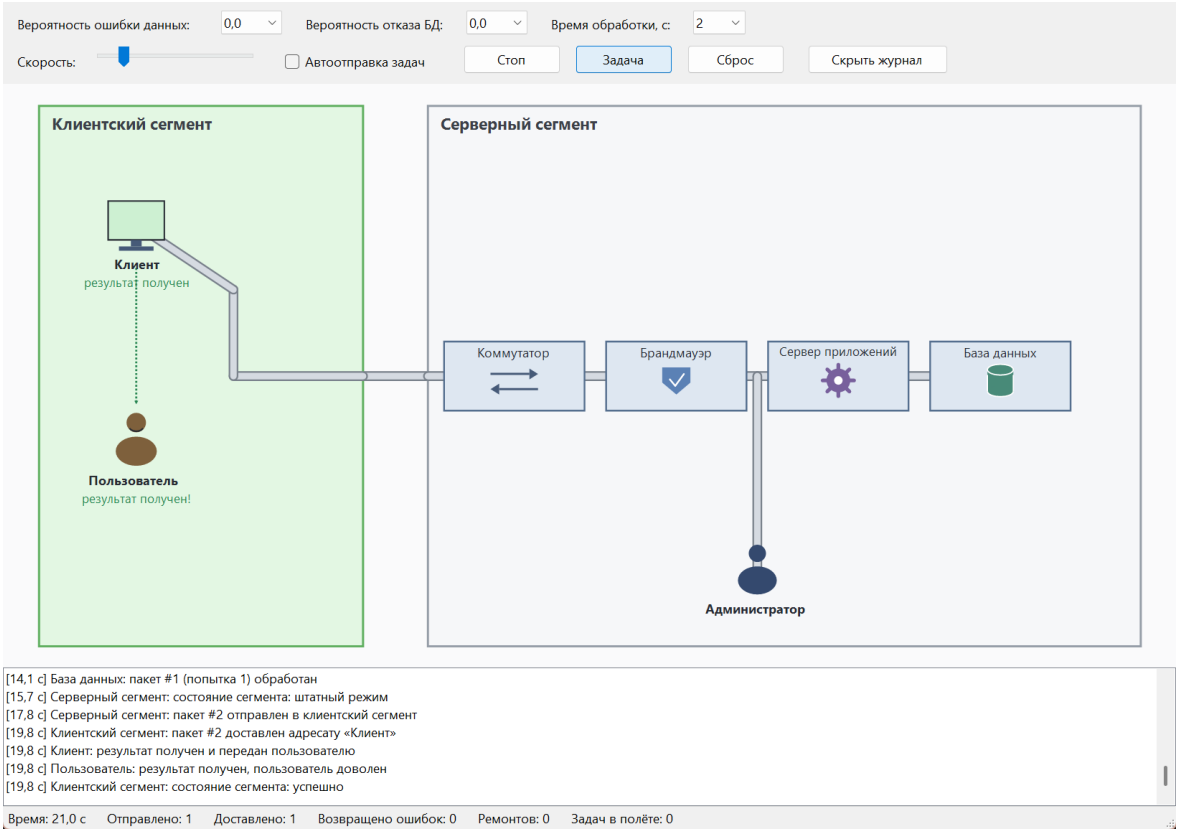



Рисунок 4 – Сценарий успешной обработки задачи

Сценарий Б (ошибка данных). Брандмауэр или сервер приложений возвращает исход InvalidData. Узел остаётся исправным, серверный сегмент создаёт пакет DataError с адресатом-владельцем задачи, пакет уходит клиенту. Клиент исправляет ошибку за FixTime и отправляет задачу заново с начала цепочки, увеличивая номер попытки.

Сценарий В (отказ узла). База данных возвращает исход NodeOutage, переходит в состояние «Отказ» и удерживает пакет. Серверный сегмент создаёт пакет NodeFailure, администратор подключается к узлу, ждёт RepairTime и отправляет команду Fix. Когда команда доходит до узла, тот ремонтируется и продолжает обработку удерживаемого пакета.

Если несколько задач приходят одновременно, они встают в очередь узла. Очередь отображается под узлом в виде лотка с пакетами (рисунок 5): пакеты стоят друг за другом и обрабатываются по одному.

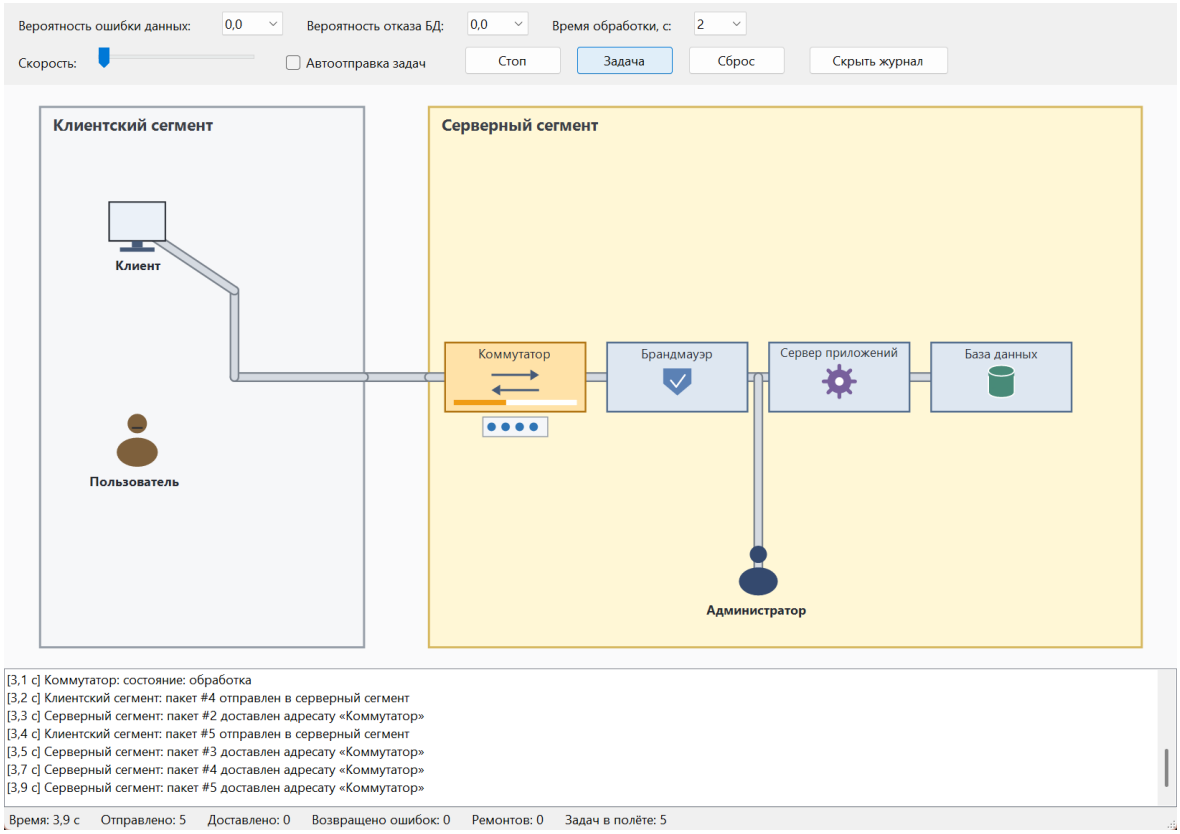



Рисунок 5 – Очередь пакетов под коммутатором

# 3 СХЕМЫ АЛГОРИТМОВ

Схемы выполнены по ГОСТ 19.701-90. Использованы символы: терминатор (начало и конец), процесс, решение, предопределённый процесс (вызов метода), данные и соединители для разрыва линий. Направление потока — сверху вниз, выходы решений подписаны «Да» и «Нет».

## 3.1 Обработка шага модельного времени узлом

Алгоритм метода OnTick и завершения обработки пакета приведён на рисунке 6. Если узел не обрабатывает пакет, шаг времени игнорируется. Иначе уменьшается оставшееся время; когда оно становится неположительным, определяется исход. После ремонта исход гарантированно успешен, иначе вызывается Evaluate. При отказе узла пакет остаётся внутри, состояние становится «Отказ» и вызывается событие ProcessingFailed; в остальных случаях узел освобождается и запускает следующий пакет из очереди.

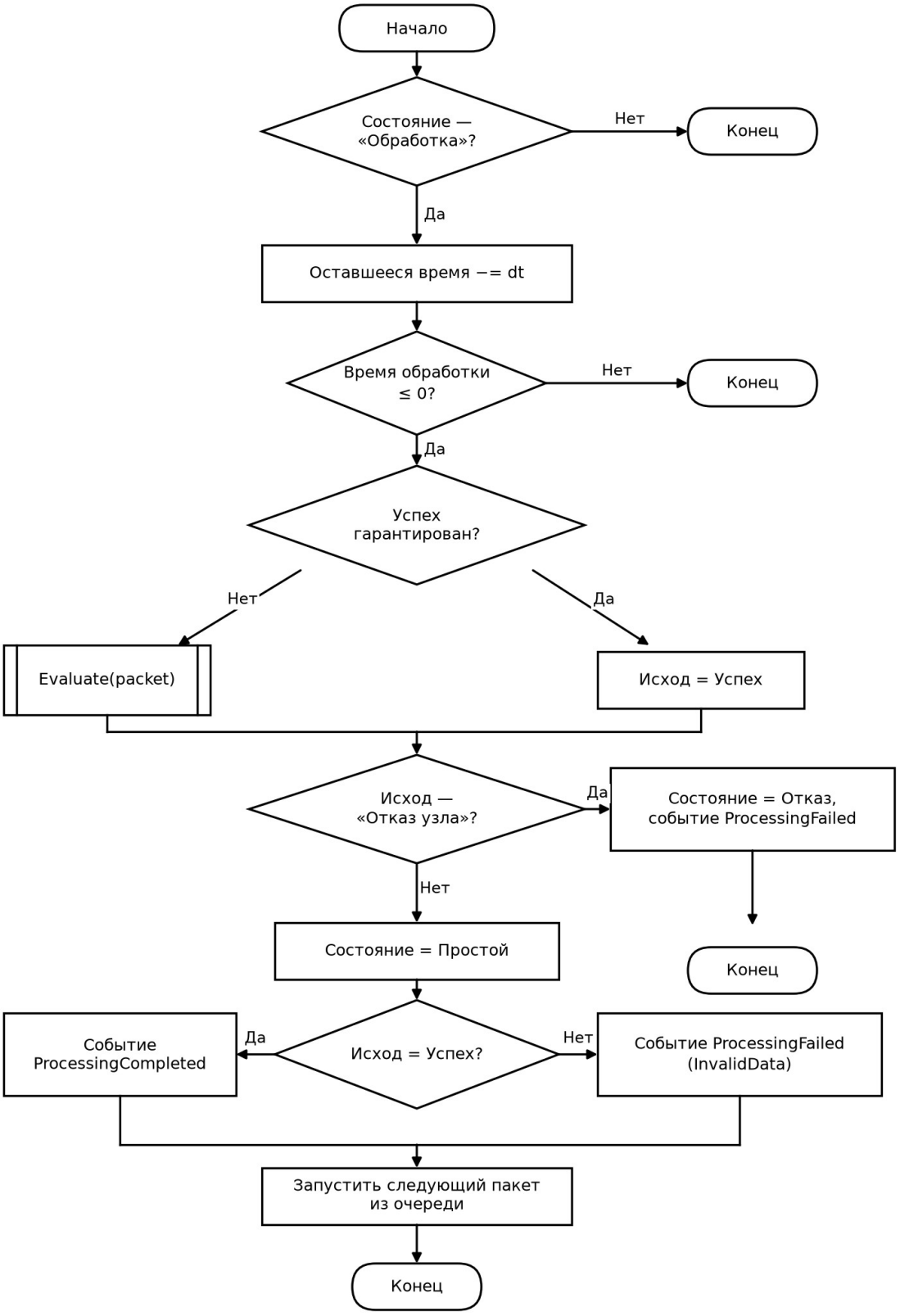



Рисунок 6 – Алгоритм обработки шага модельного времени узлом

## 3.2 Ремонт узла

Метод Repair (рисунок 7) выполняется только для отказавшего узла: он устанавливает признак гарантированного успеха, сбрасывает время обработки и переводит узел в состояние «Обработка». Удерживаемый пакет продолжает обрабатываться, цепочка заново не проходится.

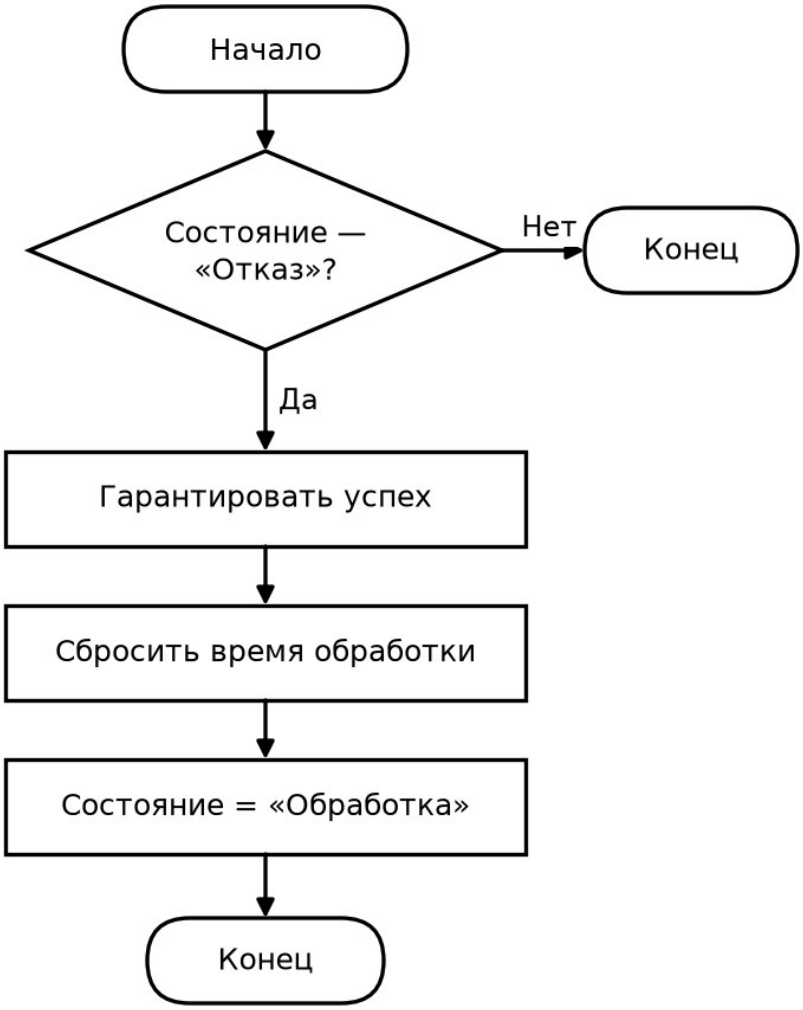



Рисунок 7 – Алгоритм ремонта узла

### 3.3 Маршрутизация пакета сегментом

Метод Submit (рисунок 8) определяет адресата пакета: если адресат не задан, вызывается ResolveDestination, переопределённый в наследниках. Для локального адресата строится маршрут по графу дорожек, для внешнего — маршрут к шлюзу и добавление точки шлюза соседнего сегмента. Если маршрут не найден, пакет отбрасывается с событием PacketDropped.

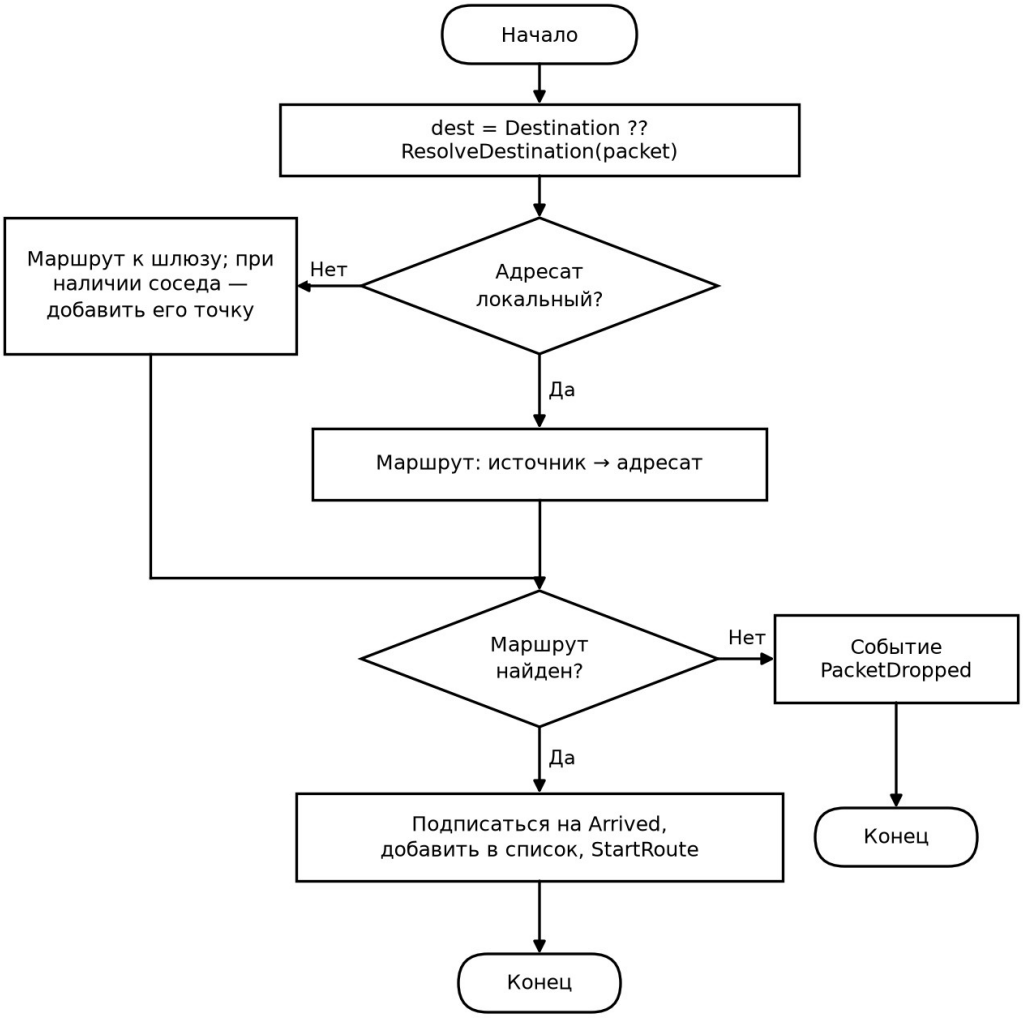



Рисунок 8 – Алгоритм маршрутизации пакета сегментом

### 3.4 Обработка отказа узла серверным сегментом

Обработчик OnNodeFailed (рисунок 9) различает вид сбоя. При ошибке данных создаётся пакет DataError с адресатом-владельцем задачи, при отказе узла — пакет NodeFailure с указанием отказавшего узла; адресата второго пакета определяет ResolveDestination (администратор).

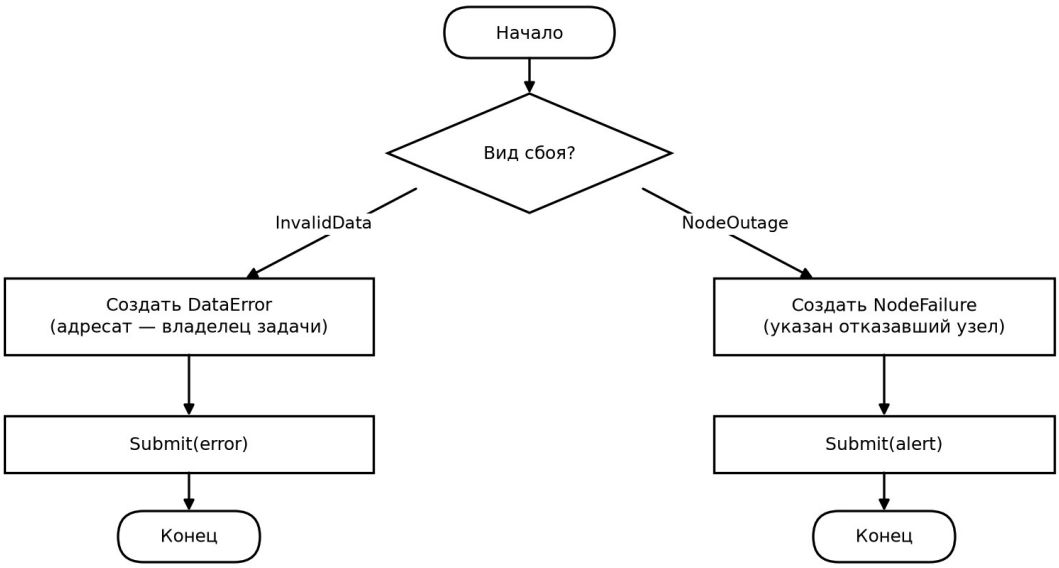



Рисунок 9 – Алгоритм обработки сбоя узла серверным сегментом

## 3.5 Исправление ошибки данных клиентом

Клиент принимает только пакеты DataError (рисунок 10). Ошибка встаёт в очередь исправлений; если клиент свободен, запускается таймер FixTime. По истечении таймера берётся очередная ошибка, проверяется предел попыток и отправляется задача с новым номером попытки. Если в очереди есть ещё ошибки, исправление продолжается.

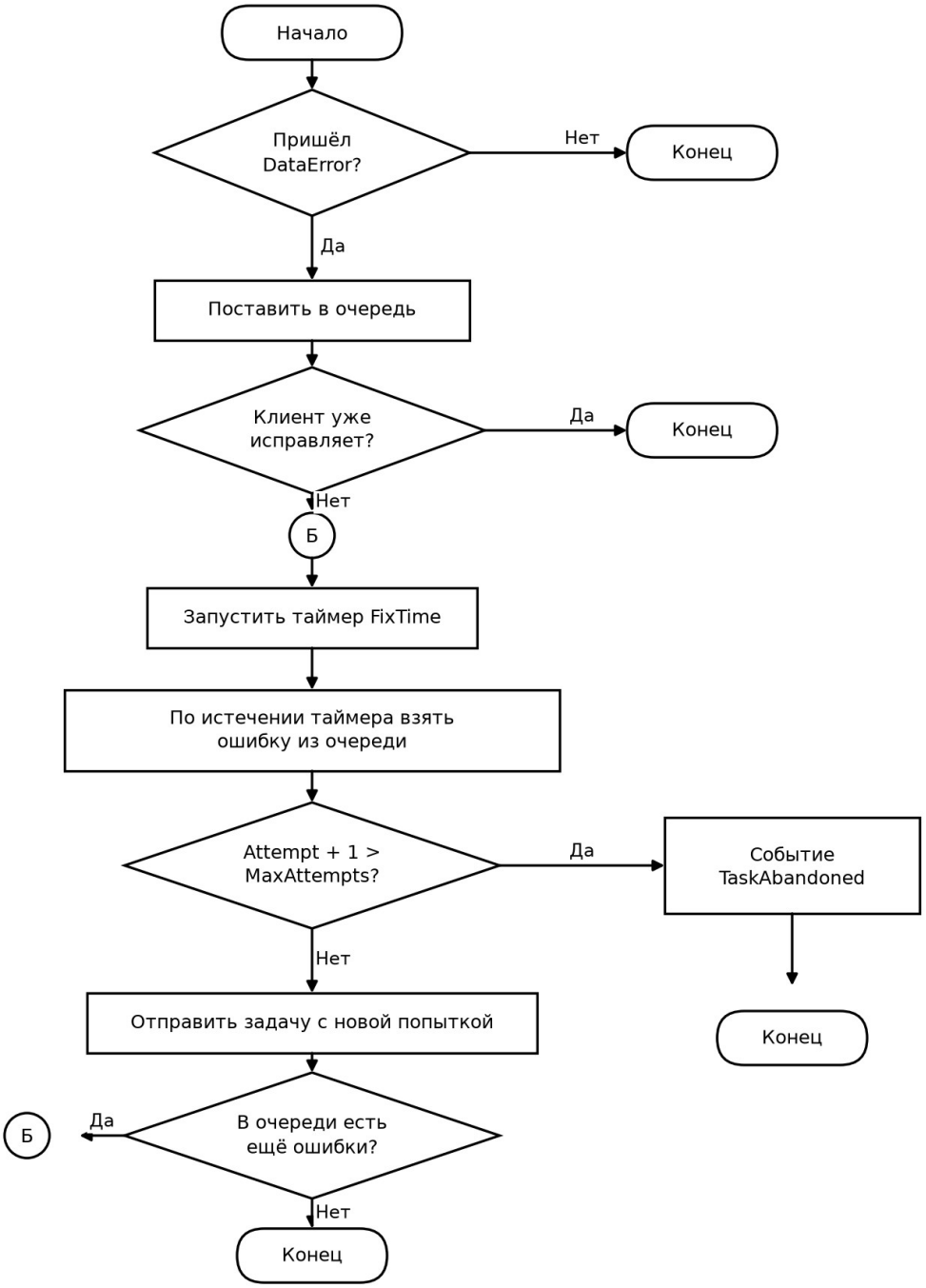



Рисунок 10 – Алгоритм исправления ошибки данных клиентом

## 3.6 Движение пакета по маршруту

Метод Advance (рисунок 11) продвигает пакет на расстояние Speed × dt. Пока оставшегося расстояния хватает, пакет переходит к следующей точке маршрута; на последнем отрезке он сдвигается частично. Когда маршрут пройден, пакет вызывает событие Arrived.

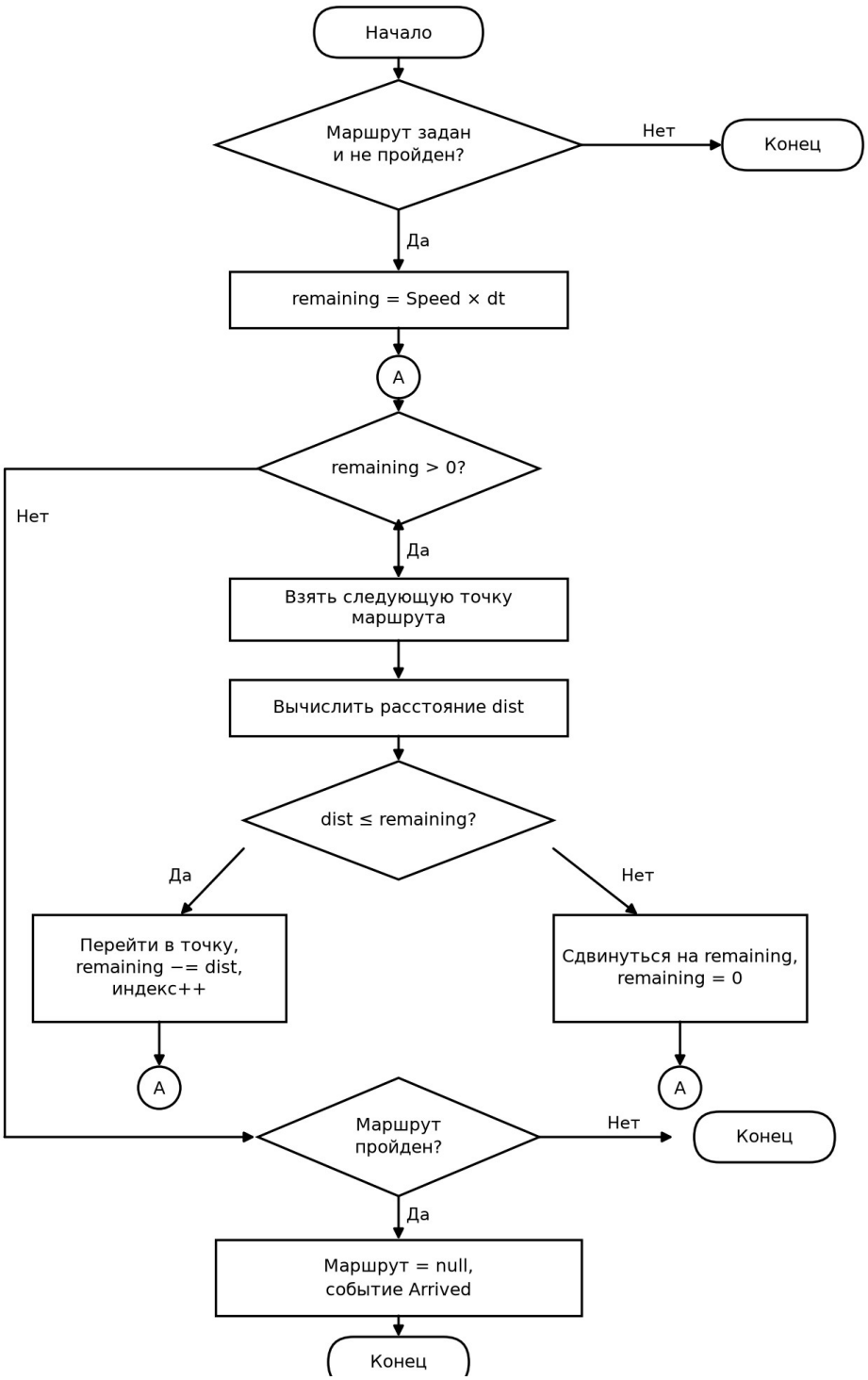



Рисунок 11 – Алгоритм движения пакета по маршруту

# 4 ТЕКСТ ПРОГРАММЫ

Ниже приведён текст программы. Код дизайнера главной формы (MainForm.Designer.cs), созданный средой Visual Studio автоматически, и файл Program.cs с точкой входа не приводятся. Полный проект находится в репозитории и на носителе.

**Файл Enums.cs**

```
namespace NetworkSim.Model
{
    /// <summary>Тип пакета, перемещающегося по сети.</summary>
    public enum PacketKind
    {
        Task,           // задача на обработку
        DataError,      // сообщение об ошибке данных (возвращается клиенту)
        NodeFailure,    // уведомление об отказе узла (направляется администратору)
        Fix,            // команда ремонта узла (от администратора)
        Result          // результат обработки задачи
    }

    /// <summary>Состояние узла обработки.</summary>
    public enum NodeStatus
    {
        Idle,           // узел свободен
        Processing,     // узел обрабатывает пакет
        Failed          // отказ: узел неисправен и ожидает ремонта
    }

    /// <summary>Вид сбоя при обработке пакета.</summary>
    public enum FailureKind
    {
        None,           // сбоя не было
        InvalidData,    // ошибка данных: пакет некорректен, узел исправен
        NodeOutage      // отказ узла: требуется ремонт
    }

    /// <summary>Состояние сегмента сети (определяет его внешний вид).</summary>
    public enum SegmentState
    {
        Normal,         // штатный режим
        Busy,           // сегмент занят обработкой
        Alert,          // аварийная ситуация
        Success         // успешное завершение задачи
    }

    /// <summary>Состояние клиента.</summary>
    public enum ClientState
    {
        Idle,           // свободен
        Fixing,         // исправляет ошибку данных
        Done            // получил результат задачи
    }

    /// <summary>Состояние администратора.</summary>
    public enum AdminState
    {
        Idle,           // ремонтов не требуется
        Repairing       // ремонтирует узел (подключён по SSH)
    }

    /// <summary>Настроение получателя результата.</summary>
```

```
    public enum UserMood
    {
        Waiting,        // ждёт результата
        Happy           // результат получен
    }
}
```

**Файл EventArgsTypes.cs**

```
using System;

namespace NetworkSim.Model
{
    /// <summary>Аргументы события «шаг модельного времени».</summary>
    public class TickEventArgs : EventArgs
    {
        public double Dt { get; }

        public TickEventArgs(double dt)
        {
            Dt = dt;
        }
    }

    /// <summary>Аргументы событий, связанных с пакетом.</summary>
    public class PacketEventArgs : EventArgs
    {
        public Packet Packet { get; }

        public PacketEventArgs(Packet packet)
        {
            Packet = packet;
        }
    }

    /// <summary>Аргументы событий, связанных с узлом обработки.</summary>
    public class NodeEventArgs : EventArgs
    {
        public ProcessingNode Node { get; }

        public Packet Packet { get; }

        public NodeEventArgs(ProcessingNode node, Packet packet)
        {
            Node = node;
            Packet = packet;
        }
    }

    /// <summary>Аргументы события «сбой при обработке пакета».</summary>
    public class NodeFailureEventArgs : NodeEventArgs
    {
        public FailureKind Kind { get; }

        public NodeFailureEventArgs(ProcessingNode node, Packet packet, FailureKind kind)
            : base(node, packet)
        {
            Kind = kind;
        }
    }

    /// <summary>Аргументы события «изменилось значение свойства».</summary>
    public class ValueChangedEventArgs<T> : EventArgs
    {
        public T OldValue { get; }
```

```
        public T NewValue { get; }

        public ValueChangedEventArgs(T oldValue, T newValue)
        {
            OldValue = oldValue;
            NewValue = newValue;
        }
    }
}
```

**Файл IPacketEndpoint.cs**

```
using System.Drawing;

namespace NetworkSim.Model
{
    /// <summary>
    /// Объект сети, способный принимать пакеты (клиент, узел обработки,
    /// администратор, получатель результата).
    /// </summary>
    public interface IPacketEndpoint
    {
        /// <summary>Имя объекта для отображения и журнала.</summary>
        string Name { get; }

        /// <summary>Идентификатор вершины графа дорожек своего сегмента.</summary>
        string WaypointId { get; }

        /// <summary>Координаты объекта на схеме.</summary>
        PointF Location { get; }

        /// <summary>Принимает пакет, дошедший до адресата.</summary>
        void Receive(Packet packet);
    }
}
```

**Файл IDrawable.cs**

```
using System.Drawing;

namespace NetworkSim.Model
{
    /// <summary>Объект, умеющий изображать себя на схеме средствами GDI+.</summary>
    public interface IDrawable
    {
        /// <summary>Рисует объект на схеме.</summary>
        void Draw(Graphics g);
    }
}
```

**Файл SimulationClock.cs**

```
using System;
using System.Diagnostics;
using System.Windows.Forms;

namespace NetworkSim.Model
{
    /// <summary>
    /// Единственный источник времени в модели: обёртка над системным таймером,
    /// порождающая событие Tick с шагом модельного времени. Все объекты модели
    /// подписываются на это событие и ведут свои таймеры накоплением шага Dt.
    /// Шаг пропорционален реально прошедшему времени, поэтому коэффициент
    /// ускорения точно задаёт скорость модели.
    /// </summary>
```

```
public class SimulationClock
{
    /// <summary>Минимальный коэффициент ускорения.</summary>
    public const double MinSpeedFactor = 0.25;

    /// <summary>Максимальный коэффициент ускорения.</summary>
    public const double MaxSpeedFactor = 5.0;

    private readonly Timer _timer;              // системный таймер (UI-поток)
    private readonly Stopwatch _stopwatch = new Stopwatch();
    private double _elapsed;                    // накопленное модельное время, с
    private double _speedFactor = 1.0;

    /// <summary>Интервал системного таймера, мс.</summary>
    public int Interval
    {
        get { return _timer.Interval; }
        set { _timer.Interval = value; }
    }

    /// <summary>Коэффициент ускорения модельного времени (от 0,25 до 5).</summary>
    public double SpeedFactor
    {
        get { return _speedFactor; }
        set
        {
            if (value < MinSpeedFactor || value > MaxSpeedFactor)
            {
                throw new ArgumentOutOfRangeException(nameof(value),
                    "Коэффициент скорости должен быть в диапазоне от 0,25 до 5.");
            }
            _speedFactor = value;
        }
    }

    /// <summary>Идёт ли моделирование в данный момент (только чтение).</summary>
    public bool IsRunning
    {
        get { return _timer.Enabled; }
    }

    /// <summary>Модельное время с момента создания часов, с (только чтение).</summary>
    public double Elapsed
    {
        get { return _elapsed; }
    }

    /// <summary>Событие: очередной шаг модельного времени.</summary>
    public event EventHandler<TickEventArgs> Tick;

    /// <summary>
    /// Создаёт часы. Интервал 25 мс система округляет до ~31 мс,
    /// что даёт около 30 шагов в секунду.
    /// </summary>
    public SimulationClock(int intervalMs = 25)
    {
        _timer = new Timer();
        _timer.Interval = intervalMs;
        _timer.Tick += OnTimerTick;
    }

    /// <summary>Запускает моделирование.</summary>
    public void Start()
    {
        _stopwatch.Restart();
        _timer.Start();
    }

    /// <summary>Останавливает моделирование.</summary>
```

```
        public void Stop()
        {
            _timer.Stop();
            _stopwatch.Reset();
        }

        /// <summary>Вызывает событие Tick.</summary>
        protected virtual void OnTick(TickEventArgs e)
        {
            Tick?.Invoke(this, e);
        }

        // Обработчик таймера: реально прошедшее время переводится в модельное.
        private void OnTimerTick(object sender, EventArgs e)
        {
            double realDt = _stopwatch.Elapsed.TotalSeconds;
            _stopwatch.Restart();
            double dt = realDt * _speedFactor;
            _elapsed += dt;
            OnTick(new TickEventArgs(dt));
        }
    }
}
```

**Файл TrackGraph.cs**

```
using System;
using System.Collections.Generic;
using System.Drawing;
using System.Drawing.Drawing2D;

namespace NetworkSim.Model
{
    /// <summary>
    /// Граф дорожек сегмента сети: вершины (точки на схеме) и соединяющие их
    /// дорожки, по которым движутся пакеты. Умеет находить маршрут между
    /// вершинами поиском в ширину и рисовать дорожки.
    /// </summary>
    public class TrackGraph
    {
        private readonly Dictionary<string, PointF> _points = new Dictionary<string, PointF>();
        private readonly Dictionary<string, List<string>> _adjacency =
            new Dictionary<string, List<string>>();
        private readonly List<KeyValuePair<string, string>> _lanes =
            new List<KeyValuePair<string, string>>();

        // Цвета дорожек (общедоступны: линия связи между сегментами та же).
        public static readonly Color CasingColor = Color.FromArgb(122, 130, 140);
        public static readonly Color RoadColor = Color.FromArgb(213, 218, 225);

        /// <summary>Добавляет вершину графа (точку на схеме сегмента).</summary>
        public void AddWaypoint(string id, PointF point)
        {
            if (!_points.ContainsKey(id))
            {
                _points[id] = point;
            }
        }

        /// <summary>
        /// Добавляет двустороннюю дорожку между двумя вершинами. Вершины
        /// могут быть добавлены и позже (до первого поиска маршрута).
        /// </summary>
        public void AddLane(string idA, string idB)
        {
            if (idA == idB)
            {
                return;
            }
```

```
            AddNeighbor(idA, idB);
            AddNeighbor(idB, idA);
            _lanes.Add(new KeyValuePair<string, string>(idA, idB));
        }

        /// <summary>Возвращает координаты вершины графа.</summary>
        public PointF GetPoint(string id)
        {
            return _points[id];
        }

        /// <summary>
        /// Ищет кратчайший маршрут между вершинами поиском в ширину.
        /// Возвращает последовательность точек или null, если пути нет.
        /// </summary>
        public List<PointF> FindRoute(string fromId, string toId)
        {
            if (!_points.ContainsKey(fromId) || !_points.ContainsKey(toId))
            {
                return null;
            }
            List<PointF> route = new List<PointF>();
            if (fromId == toId)
            {
                route.Add(_points[fromId]);
                return route;
            }

            // Поиск в ширину: previous хранит вершину, из которой пришли.
            Dictionary<string, string> previous = new Dictionary<string, string>();
            Queue<string> frontier = new Queue<string>();
            frontier.Enqueue(fromId);
            previous[fromId] = null;
            bool found = false;
            while (frontier.Count > 0 && !found)
            {
                string current = frontier.Dequeue();
                List<string> neighbors;
                if (!_adjacency.TryGetValue(current, out neighbors))
                {
                    continue;
                }
                foreach (string next in neighbors)
                {
                    if (previous.ContainsKey(next))
                    {
                        continue;
                    }
                    previous[next] = current;
                    if (next == toId)
                    {
                        found = true;
                        break;
                    }
                    frontier.Enqueue(next);
                }
            }
            if (!found)
            {
                return null;
            }

            // Восстановление пути: от конечной вершины поднимаемся к начальной.
            Stack<string> path = new Stack<string>();
            string step = toId;
            while (step != null)
            {
                path.Push(step);
                step = previous[step];
            }
            foreach (string id in path)
```

```
            {
                route.Add(_points[id]);
            }
            return route;
        }

        /// <summary>Рисует дорожки сегмента толстыми серыми линиями.</summary>
        public void Draw(Graphics g)
        {
            using (Pen casing = new Pen(CasingColor, 9f))
            using (Pen road = new Pen(RoadColor, 6f))
            {
                casing.StartCap = LineCap.Round;
                casing.EndCap = LineCap.Round;
                road.StartCap = LineCap.Round;
                road.EndCap = LineCap.Round;
                foreach (KeyValuePair<string, string> lane in _lanes)
                {
                    PointF a, b;
                    if (!_points.TryGetValue(lane.Key, out a) ||
                        !_points.TryGetValue(lane.Value, out b))
                    {
                        continue;   // вершина ещё не зарегистрирована
                    }
                    g.DrawLine(casing, a, b);
                    g.DrawLine(road, a, b);
                }
            }
        }

        // Добавляет вершине соседа по дорожке (без повторов).
        private void AddNeighbor(string fromId, string toId)
        {
            List<string> neighbors;
            if (!_adjacency.TryGetValue(fromId, out neighbors))
            {
                neighbors = new List<string>();
                _adjacency[fromId] = neighbors;
            }
            if (!neighbors.Contains(toId))
            {
                neighbors.Add(toId);
            }
        }
    }
}
```

**Файл Packet.cs**

```
using System;
using System.Collections.Generic;
using System.Drawing;

namespace NetworkSim.Model
{
    /// <summary>
    /// Пакет — перемещающийся по дорожкам сети объект. Вид пакета определяет
    /// его назначение: задача, ошибка данных, уведомление об отказе узла,
    /// команда ремонта или результат обработки.
    /// </summary>
    public class Packet : IDrawable
    {
        private static int _nextId = 1;     // общий счётчик номеров пакетов

        private const float DefaultSpeed = 160f;    // скорость, пикселей схемы в секунду

        private List<PointF> _route;        // текущий маршрут (null, если пакет не движется)
        private int _routeIndex;            // индекс следующей точки маршрута
        private PointF _position;           // текущая позиция на схеме
```

```
        private static readonly Brush TaskBrush = new SolidBrush(Color.FromArgb(46, 116, 181));
        private static readonly Brush ErrorBrush = new SolidBrush(Color.FromArgb(198, 40, 40));
        private static readonly Brush FailureBrush = new SolidBrush(Color.FromArgb(230, 126, 34));
        private static readonly Brush FixBrush = new SolidBrush(Color.FromArgb(39, 134, 82));
        private static readonly Brush ResultBrush = new SolidBrush(Color.FromArgb(212, 160, 23));
        private static readonly Pen OutlinePen = new Pen(Color.White, 1.4f);
        private static readonly Brush WhiteBrush = new SolidBrush(Color.White);
        private static readonly Font MarkerFont = new Font("Arial", 9f, FontStyle.Bold,
GraphicsUnit.Pixel);

        /// <summary>Размер значка пакета на схеме, пикселей.</summary>
        public const float VisualSize = 13f;

        public int Id { get; }

        public PacketKind Kind { get; }

        public IPacketEndpoint Owner { get; }

        public int Attempt { get; }

        public ProcessingNode Subject { get; }

        /// <summary>Адресат пакета; null означает «за пределами сегмента».</summary>
        public IPacketEndpoint Destination { get; set; }

        /// <summary>Идентификатор вершины графа, из которой пакет начинает движение.</summary>
        public string SourceWaypointId { get; private set; }

        /// <summary>Текущая позиция пакета на схеме.</summary>
        public PointF Position
        {
            get { return _position; }
        }

        /// <summary>Скорость движения, пикселей схемы в секунду.</summary>
        public double Speed { get; set; }

        /// <summary>Событие: пакет дошёл до конца маршрута.</summary>
        public event EventHandler<PacketEventArgs> Arrived;

        /// <summary>Создаёт пакет заданного типа.</summary>
        public Packet(PacketKind kind, string sourceWaypointId, IPacketEndpoint owner,
                      int attempt, ProcessingNode subject = null)
        {
            Id = _nextId++;
            Kind = kind;
            SourceWaypointId = sourceWaypointId;
            Owner = owner;
            Attempt = attempt;
            Subject = subject;
            Speed = DefaultSpeed;
        }

        /// <summary>
        /// Перенаправляет пакет: задаёт нового адресата и вершину,
        /// из которой пакет продолжит движение.
        /// </summary>
        public void Readdress(IPacketEndpoint destination, string fromWaypointId)
        {
            Destination = destination;
            SourceWaypointId = fromWaypointId;
            _route = null;
            _routeIndex = 0;
        }
```

```
        /// <summary>Запускает пакет по маршруту (последовательности точек схемы).</summary>
        public void StartRoute(IList<PointF> route)
        {
            if (route == null || route.Count == 0)
            {
                return;
            }
            _route = new List<PointF>(route);
            _position = _route[0];
            _routeIndex = 1;
        }

        /// <summary>
        /// Продвигает пакет по маршруту за шаг модельного времени dt.
        /// Когда маршрут пройден полностью, вызывает событие Arrived.
        /// </summary>
        public void Advance(double dt)
        {
            if (_route == null || _routeIndex >= _route.Count)
            {
                return;     // пакет не движется
            }
            float remaining = (float)(Speed * dt);
            while (remaining > 0f && _routeIndex < _route.Count)
            {
                PointF target = _route[_routeIndex];
                float dx = target.X - _position.X;
                float dy = target.Y - _position.Y;
                float dist = (float)Math.Sqrt(dx * dx + dy * dy);
                if (dist <= remaining)
                {
                    // Доходим до очередной точки маршрута и переходим к следующей.
                    _position = target;
                    remaining -= dist;
                    _routeIndex++;
                }
                else
                {
                    // Сдвигаемся к следующей точке на оставшуюся часть шага.
                    _position = new PointF(
                        _position.X + dx / dist * remaining,
                        _position.Y + dy / dist * remaining);
                    remaining = 0f;
                }
            }
            if (_routeIndex >= _route.Count)
            {
                _route = null;      // маршрут пройден полностью
                OnArrived(new PacketEventArgs(this));
            }
        }

        /// <summary>Рисует пакет; форма и цвет зависят от типа пакета.</summary>
        public void Draw(Graphics g)
        {
            float s = VisualSize;
            float x = _position.X - s / 2f;
            float y = _position.Y - s / 2f;
            switch (Kind)
            {
                case PacketKind.Task:
                    g.FillEllipse(TaskBrush, x, y, s, s);
                    g.DrawEllipse(OutlinePen, x, y, s, s);
                    DrawAttemptNumber(g);       // номер попытки внутри пакета
                    break;

                case PacketKind.DataError:
                    // Красный треугольник — ошибка данных.
                    PointF[] triangle =
                    {
                        new PointF(x + s / 2f, y),
```

```
                        new PointF(x + s, y + s),
                        new PointF(x, y + s)
                    };
                    g.FillPolygon(ErrorBrush, triangle);
                    g.DrawPolygon(OutlinePen, triangle);
                    break;

                case PacketKind.NodeFailure:
                    // Оранжевый ромб с восклицательным знаком — отказ узла.
                    PointF[] diamond =
                    {
                        new PointF(x + s / 2f, y),
                        new PointF(x + s, y + s / 2f),
                        new PointF(x + s / 2f, y + s),
                        new PointF(x, y + s / 2f)
                    };
                    g.FillPolygon(FailureBrush, diamond);
                    g.DrawPolygon(OutlinePen, diamond);
                    SizeF excl = g.MeasureString("!", MarkerFont);
                    g.DrawString("!", MarkerFont, WhiteBrush,
                        _position.X - excl.Width / 2f, _position.Y - excl.Height / 2f);
                    break;

                case PacketKind.Fix:
                    // Зелёный квадрат — команда ремонта.
                    g.FillRectangle(FixBrush, x, y, s, s);
                    g.DrawRectangle(OutlinePen, x, y, s, s);
                    break;

                case PacketKind.Result:
                    // Золотой круг с галочкой — результат.
                    g.FillEllipse(ResultBrush, x, y, s, s);
                    g.DrawEllipse(OutlinePen, x, y, s, s);
                    PointF[] check =
                    {
                        new PointF(x + 3f, y + 7f),
                        new PointF(x + 5.5f, y + 9.5f),
                        new PointF(x + 10f, y + 3.5f)
                    };
                    g.DrawLines(OutlinePen, check);
                    break;
            }
        }

        /// <summary>Вызывает событие Arrived.</summary>
        protected virtual void OnArrived(PacketEventArgs e)
        {
            Arrived?.Invoke(this, e);
        }

        // Рисует номер попытки выполнения задачи внутри кружка задачи.
        private void DrawAttemptNumber(Graphics g)
        {
            string text = Attempt.ToString();
            SizeF size = g.MeasureString(text, MarkerFont);
            g.DrawString(text, MarkerFont, WhiteBrush,
                _position.X - size.Width / 2f, _position.Y - size.Height / 2f);
        }
    }
}
```

**Файл ProcessingNode.cs**

```
using System;
using System.Collections.Generic;
using System.Drawing;
```

```
namespace NetworkSim.Model
{
    /// <summary>
    /// Делегат события «узел завершил обработку пакета». Объявлен явно,
    /// а не взят готовым EventHandler, как требует задание.
    /// </summary>
    public delegate void NodeCompletedHandler(object sender, NodeEventArgs e);

    /// <summary>
    /// Делегат события «узел не смог обработать пакет». Объявлен явно,
    /// а не взят готовым EventHandler, как требует задание.
    /// </summary>
    public delegate void NodeFailedHandler(object sender, NodeFailureEventArgs e);

    /// <summary>
    /// Основной класс модели: узел обработки пакетов. Принимает задачи,
    /// обрабатывает их заданное время и сообщает о результате событиями
    /// ProcessingCompleted и ProcessingFailed. Может выйти из строя и быть
    /// отремонтирован. Является базовым классом для специализированных узлов.
    /// </summary>
    public class ProcessingNode : IPacketEndpoint, IDrawable
    {
        private static int _nextIndex = 1;          // общий счётчик номеров узлов

        private readonly Queue<Packet> _queue = new Queue<Packet>();   // ожидающие задачи
        private Packet _currentPacket;              // обрабатываемый (или удерживаемый) пакет
        private double _processingRemaining;        // оставшееся время обработки, с
        private bool _successGuaranteed;            // после ремонта следующая обработка успешна
        private NodeStatus _status = NodeStatus.Idle;
        private double _processingTime;
        private double _failureProbability;

        /// <summary>Часы моделирования, на шаги которых подписан узел.</summary>
        protected readonly SimulationClock Clock;

        /// <summary>Ширина узла на схеме.</summary>
        public const float NodeWidth = 130f;

        /// <summary>Высота узла на схеме.</summary>
        public const float NodeHeight = 64f;

        private const int QueueSlot = 13;           // шаг между пакетами в очереди
        private const int QueueMaxVisible = 5;      // сколько пакетов очереди рисуем
        private const int LongNameLength = 14;      // с какой длины название пишется мельче

        // Кисти, перья и шрифты для рисования (создаются один раз).
        private static readonly Brush IdleBrush = new SolidBrush(Color.FromArgb(222, 231, 241));
        private static readonly Brush ProcessingBrush = new SolidBrush(Color.FromArgb(255, 226,
168));
        private static readonly Brush FailedBrush = new SolidBrush(Color.FromArgb(245, 183, 183));
        private static readonly Pen IdlePen = new Pen(Color.FromArgb(84, 110, 142), 1.6f);
        private static readonly Pen ProcessingPen = new Pen(Color.FromArgb(176, 116, 18), 1.6f);
        private static readonly Pen FailedPen = new Pen(Color.FromArgb(158, 42, 42), 1.8f);
        private static readonly Pen QueuePen = new Pen(Color.FromArgb(150, 158, 168), 1f);
        private static readonly Brush NameBrush = new SolidBrush(Color.FromArgb(40, 44, 52));
        private static readonly Brush QueueBackBrush = new SolidBrush(Color.FromArgb(244, 246,
249));
        private static readonly Brush QueuePacketBrush = new SolidBrush(Color.FromArgb(46, 116,
181));
        private static readonly Brush ProgressBackBrush = new SolidBrush(Color.White);
        private static readonly Brush ProgressBrush = new SolidBrush(Color.FromArgb(240, 156, 20));
        private static readonly Brush QueueBrush = new SolidBrush(Color.FromArgb(110, 116, 126));
        private static readonly Brush CrossBrush = new SolidBrush(Color.FromArgb(158, 42, 42));
        private static readonly Brush IconBrush = new SolidBrush(Color.FromArgb(70, 90, 120));

        // Шрифты заданы в пикселях: их размер не зависит от масштаба экрана.
        private static readonly Font NameFont = new Font("Segoe UI", 12f, FontStyle.Regular,
GraphicsUnit.Pixel);
```

```
        private static readonly Font LongNameFont = new Font("Segoe UI", 11f, FontStyle.Regular,
GraphicsUnit.Pixel);
        private static readonly Font QueueFont = new Font("Segoe UI", 10f, FontStyle.Regular,
GraphicsUnit.Pixel);
        private static readonly Font CrossFont = new Font("Arial", 22f, FontStyle.Bold,
GraphicsUnit.Pixel);

        // Выравнивание названия по центру рамки.
        private static readonly StringFormat NameFormat = new StringFormat
        {
            Alignment = StringAlignment.Center,
            LineAlignment = StringAlignment.Near
        };

        public int Index { get; }

        public string Name { get; }

        public string WaypointId { get; }

        public PointF Location { get; }

        /// <summary>Состояние узла: простой / обработка / отказ (снаружи только чтение).</summary>
        public NodeStatus Status
        {
            get { return _status; }
            private set
            {
                if (_status == value)
                {
                    return;
                }
                NodeStatus old = _status;
                _status = value;
                OnStatusChanged(new ValueChangedEventArgs<NodeStatus>(old, value));
            }
        }

        /// <summary>Время обработки одного пакета, с (должно быть больше нуля).</summary>
        public double ProcessingTime
        {
            get { return _processingTime; }
            set
            {
                if (value <= 0)
                {
                    throw new ArgumentOutOfRangeException(nameof(value),
                        "Время обработки должно быть положительным числом.");
                }
                _processingTime = value;
            }
        }

        /// <summary>Вероятность сбоя при обработке пакета, значение от 0 до 1.</summary>
        public double FailureProbability
        {
            get { return _failureProbability; }
            set
            {
                if (value < 0 || value > 1)
                {
                    throw new ArgumentOutOfRangeException(nameof(value),
                        "Вероятность сбоя должна быть в диапазоне от 0 до 1.");
                }
                _failureProbability = value;
            }
        }

        /// <summary>Число пакетов, ожидающих обработки в очереди (только чтение).</summary>
```

```
        public int QueueLength
        {
            get { return _queue.Count; }
        }

        // Хранилище подписчиков для событий с явными add/remove.
        private NodeCompletedHandler _processingCompleted;
        private NodeFailedHandler _processingFailed;

        /// <summary>
        /// Событие: обработка пакета успешно завершена. Объявлено полностью,
        /// с явными методами add и remove (а не «по умолчанию»).
        /// </summary>
        public event NodeCompletedHandler ProcessingCompleted
        {
            add { _processingCompleted += value; }
            remove { _processingCompleted -= value; }
        }

        /// <summary>
        /// Событие: обработка завершилась ошибкой данных или отказом узла.
        /// Объявлено полностью, с явными методами add и remove.
        /// </summary>
        public event NodeFailedHandler ProcessingFailed
        {
            add { _processingFailed += value; }
            remove { _processingFailed -= value; }
        }

        /// <summary>Событие: изменилось состояние узла.</summary>
        public event EventHandler<ValueChangedEventArgs<NodeStatus>> StatusChanged;

        /// <summary>Создаёт узел обработки с заданными параметрами.</summary>
        public ProcessingNode(string name, string waypointId, PointF location,
                              SimulationClock clock, double processingTime = 1.5,
                              double failureProbability = 0.0)
        {
            Index = _nextIndex++;
            Name = name;
            WaypointId = waypointId;
            Location = location;
            Clock = clock;
            ProcessingTime = processingTime;        // через свойство — с проверкой
            FailureProbability = failureProbability;
            clock.Tick += OnTick;
        }

        /// <summary>
        /// Принимает пакет: задача ставится в очередь, команда ремонта
        /// чинит отказавший узел.
        /// </summary>
        public void Receive(Packet packet)
        {
            if (packet == null)
            {
                return;
            }
            if (packet.Kind == PacketKind.Task)
            {
                _queue.Enqueue(packet);
                TryStartNext();
            }
            else if (packet.Kind == PacketKind.Fix)
            {
                Repair();
            }
        }

        /// <summary>
        /// Ремонтирует отказавший узел: удерживаемый пакет продолжает
```

```
        /// обрабатываться, следующая обработка гарантированно успешна.
        /// </summary>
        public void Repair()
        {
            if (Status != NodeStatus.Failed)
            {
                return;
            }
            _successGuaranteed = true;
            _processingRemaining = ProcessingTime;
            Status = NodeStatus.Processing;
        }

        /// <summary>
        /// Определяет исход обработки пакета. Базовое поведение: с вероятностью
        /// EffectiveProbability — ошибка данных, иначе успех. Переопределяется
        /// в классах-наследниках.
        /// </summary>
        protected virtual FailureKind Evaluate(Packet packet)
        {
            return Random.Shared.NextDouble() < EffectiveProbability(packet)
                ? FailureKind.InvalidData
                : FailureKind.None;
        }

        /// <summary>
        /// Действующая вероятность сбоя: базовая, уменьшенная вдвое за каждую
        /// повторную попытку доставки задачи.
        /// </summary>
        protected double EffectiveProbability(Packet packet)
        {
            double p = FailureProbability;
            int attempt = packet != null ? packet.Attempt : 1;
            while (attempt > 1)
            {
                p /= 2.0;
                attempt--;
            }
            return p;
        }

        /// <summary>Вызывает событие ProcessingCompleted.</summary>
        protected virtual void OnProcessingCompleted(NodeEventArgs e)
        {
            _processingCompleted?.Invoke(this, e);
        }

        /// <summary>Вызывает событие ProcessingFailed.</summary>
        protected virtual void OnProcessingFailed(NodeFailureEventArgs e)
        {
            _processingFailed?.Invoke(this, e);
        }

        /// <summary>Вызывает событие StatusChanged.</summary>
        protected virtual void OnStatusChanged(ValueChangedEventArgs<NodeStatus> e)
        {
            StatusChanged?.Invoke(this, e);
        }

        /// <summary>Рисует узел: корпус, очередь и пиктограмму.</summary>
        public virtual void Draw(Graphics g)
        {
            DrawBody(g);
            if (Status != NodeStatus.Failed)
            {
                DrawSwitchIcon(g);
            }
        }

        /// <summary>Рисует общую для всех узлов часть: корпус и очередь пакетов.</summary>
```

```
        protected void DrawBody(Graphics g)
        {
            DrawFrame(g);
            DrawQueue(g);
        }

        /// <summary>
        /// Рисует корпус узла: заливку и рамку по состоянию, название,
        /// полосу прогресса обработки и знак отказа.
        /// </summary>
        protected void DrawFrame(Graphics g)
        {
            RectangleF box = new RectangleF(
                Location.X - NodeWidth / 2f, Location.Y - NodeHeight / 2f,
                NodeWidth, NodeHeight);

            Brush fill;
            Pen border;
            if (Status == NodeStatus.Processing)
            {
                fill = ProcessingBrush;
                border = ProcessingPen;
            }
            else if (Status == NodeStatus.Failed)
            {
                fill = FailedBrush;
                border = FailedPen;
            }
            else
            {
                fill = IdleBrush;
                border = IdlePen;
            }

            g.FillRectangle(fill, box);
            g.DrawRectangle(border, box.X, box.Y, box.Width, box.Height);

            // Длинные названия пишутся мельче, чтобы поместиться в рамку.
            Font nameFont = Name.Length > LongNameLength ? LongNameFont : NameFont;
            g.DrawString(Name, nameFont, NameBrush,
                new RectangleF(box.X, box.Y + 2f, box.Width, 20f), NameFormat);

            if (Status == NodeStatus.Processing)
            {
                // Полоса прогресса обработки текущего пакета.
                double fraction = 1.0 - Math.Max(0, _processingRemaining) / ProcessingTime;
                if (fraction < 0) fraction = 0;
                if (fraction > 1) fraction = 1;
                RectangleF bar = new RectangleF(box.X + 8f, box.Bottom - 11f, box.Width - 16f, 5f);
                g.FillRectangle(ProgressBackBrush, bar);
                g.FillRectangle(ProgressBrush, bar.X, bar.Y, (float)(bar.Width * fraction),
bar.Height);
            }

            if (Status == NodeStatus.Failed)
            {
                // Знак отказа поверх корпуса.
                SizeF crossSize = g.MeasureString("×", CrossFont);
                g.DrawString("×", CrossFont, CrossBrush,
                    Location.X - crossSize.Width / 2f,
                    Location.Y - crossSize.Height / 2f - 2f);
            }
        }

        // Рисует очередь пакетов под узлом: они «стоят друг за другом» и ждут обработки.
        private void DrawQueue(Graphics g)
        {
            int count = _queue.Count;
            if (count == 0)
            {
```

```
            return;
        }
        int shown = Math.Min(count, QueueMaxVisible);
        float width = 8f + shown * QueueSlot + (count > shown ? 18f : 0f);
        RectangleF tray = new RectangleF(Location.X - width / 2f,
            Location.Y + NodeHeight / 2f + 5f, width, 18f);

        g.FillRectangle(QueueBackBrush, tray);
        g.DrawRectangle(Status == NodeStatus.Failed ? FailedPen : QueuePen,
            tray.X, tray.Y, tray.Width, tray.Height);

        for (int i = 0; i < shown; i++)
        {
            float cx = tray.X + 8f + i * QueueSlot;
            g.FillEllipse(QueuePacketBrush, cx - 4f, tray.Y + 5f, 8f, 8f);
        }
        if (count > shown)
        {
            g.DrawString("+" + (count - shown), QueueFont, QueueBrush,
                tray.X + 4f + shown * QueueSlot, tray.Y + 2f);
        }
    }

    // Запускает обработку следующего пакета из очереди, если узел свободен.
    private void TryStartNext()
    {
        if (Status != NodeStatus.Idle || _queue.Count == 0)
        {
            return;
        }
        _currentPacket = _queue.Dequeue();
        _processingRemaining = ProcessingTime;
        Status = NodeStatus.Processing;
    }

    // Шаг модельного времени: продвижение обработки текущего пакета.
    private void OnTick(object sender, TickEventArgs e)
    {
        if (Status != NodeStatus.Processing)
        {
            return;
        }
        _processingRemaining -= e.Dt;
        if (_processingRemaining > 0)
        {
            return;
        }
        CompleteCurrent();
    }

    // Завершает обработку пакета по автомату состояний узла.
    private void CompleteCurrent()
    {
        FailureKind verdict = _successGuaranteed ? FailureKind.None : Evaluate(_currentPacket);
        _successGuaranteed = false;

        if (verdict == FailureKind.NodeOutage)
        {
            // Узел выходит из строя; пакет удерживается до ремонта.
            Status = NodeStatus.Failed;
            OnProcessingFailed(new NodeFailureEventArgs(this, _currentPacket, verdict));
            return;
        }

        Packet completed = _currentPacket;
        _currentPacket = null;
        Status = NodeStatus.Idle;
        if (verdict == FailureKind.None)
        {
            OnProcessingCompleted(new NodeEventArgs(this, completed));
```

```
            }
            else
            {
                OnProcessingFailed(new NodeFailureEventArgs(this, completed, verdict));
            }
            TryStartNext();
        }

        // Пиктограмма базового узла (коммутатор): две встречные стрелки.
        private void DrawSwitchIcon(Graphics g)
        {
            float cx = Location.X;
            float cy = Location.Y + 4f;
            using (Pen p = new Pen(IconBrush, 2f))
            {
                g.DrawLine(p, cx - 22f, cy - 6f, cx + 14f, cy - 6f);
                g.DrawLine(p, cx - 14f, cy + 8f, cx + 22f, cy + 8f);
            }
            PointF[] headRight =
            {
                new PointF(cx + 14f, cy - 11f),
                new PointF(cx + 22f, cy - 6f),
                new PointF(cx + 14f, cy - 1f)
            };
            PointF[] headLeft =
            {
                new PointF(cx - 14f, cy + 3f),
                new PointF(cx - 22f, cy + 8f),
                new PointF(cx - 14f, cy + 13f)
            };
            g.FillPolygon(IconBrush, headRight);
            g.FillPolygon(IconBrush, headLeft);
        }
    }
}
```

**Файл FirewallNode.cs**

```
using System;
using System.Drawing;
using System.Drawing.Drawing2D;

namespace NetworkSim.Model
{
    /// <summary>
    /// Брандмауэр — узел, производный от ProcessingNode. Проверяет данные
    /// задачи; в строгом режиме вероятность блокировки удваивается.
    /// Переопределяет Evaluate и Draw базового класса.
    /// </summary>
    public class FirewallNode : ProcessingNode
    {
        private int _blockedCount;      // число заблокированных пакетов

        private static readonly Brush ShieldBrush = new SolidBrush(Color.FromArgb(91, 129, 181));

        /// <summary>Строгий режим проверки (вероятность блокировки удваивается).</summary>
        public bool StrictMode { get; set; }

        /// <summary>Число пакетов, заблокированных брандмауэром (только чтение).</summary>
        public int BlockedCount
        {
            get { return _blockedCount; }
        }

        /// <summary>Создаёт брандмауэр с заданными параметрами.</summary>
        public FirewallNode(string name, string waypointId, PointF location,
                            SimulationClock clock, double processingTime = 1.5,
                            double failureProbability = 0.1)
            : base(name, waypointId, location, clock, processingTime, failureProbability)
```

```
        {
        }

        /// <summary>
        /// Определяет исход проверки: как в базовом классе, но в строгом
        /// режиме вероятность ошибки данных удваивается.
        /// </summary>
        protected override FailureKind Evaluate(Packet packet)
        {
            double p = EffectiveProbability(packet);
            if (StrictMode)
            {
                p = Math.Min(1.0, p * 2.0);
            }
            if (Random.Shared.NextDouble() < p)
            {
                _blockedCount++;
                return FailureKind.InvalidData;
            }
            return FailureKind.None;
        }

        /// <summary>Рисует узел с пиктограммой щита.</summary>
        public override void Draw(Graphics g)
        {
            DrawBody(g);
            if (Status != NodeStatus.Failed)
            {
                DrawShieldIcon(g);
            }
        }

        // Пиктограмма щита с галочкой.
        private void DrawShieldIcon(Graphics g)
        {
            float cx = Location.X;
            float cy = Location.Y + 4f;
            PointF[] shield =
            {
                new PointF(cx - 13f, cy - 12f),
                new PointF(cx + 13f, cy - 12f),
                new PointF(cx + 13f, cy + 2f),
                new PointF(cx, cy + 13f),
                new PointF(cx - 13f, cy + 2f)
            };
            g.FillPolygon(ShieldBrush, shield);
            using (Pen p = new Pen(Color.White, 1.6f))
            {
                p.StartCap = LineCap.Round;
                p.EndCap = LineCap.Round;
                g.DrawLine(p, cx - 6f, cy - 1f, cx - 1f, cy + 5f);
                g.DrawLine(p, cx - 1f, cy + 5f, cx + 7f, cy - 6f);
            }
        }
    }
}
```

**Файл AppServerNode.cs**

```
using System;
using System.Drawing;
using System.Drawing.Drawing2D;

namespace NetworkSim.Model
{
    /// <summary>
    /// Сервер приложений — узел, производный от ProcessingNode. Вероятность
    /// ошибки данных растёт с нагрузкой (длиной очереди пакетов).
    /// Переопределяет Evaluate, OnProcessingCompleted и Draw базового класса.
```

```
/// </summary>
public class AppServerNode : ProcessingNode
{
    private int _handledCount;      // число успешно обработанных задач

    private static readonly Brush GearBrush = new SolidBrush(Color.FromArgb(120, 96, 156));
    private static readonly Brush HubBrush = new SolidBrush(Color.White);

    /// <summary>Число успешно обработанных задач (только чтение).</summary>
    public int HandledCount
    {
        get { return _handledCount; }
    }

    /// <summary>Создаёт сервер приложений с заданными параметрами.</summary>
    public AppServerNode(string name, string waypointId, PointF location,
                         SimulationClock clock, double processingTime = 2.0,
                         double failureProbability = 0.1)
        : base(name, waypointId, location, clock, processingTime, failureProbability)
    {
    }

    /// <summary>
    /// Определяет исход обработки: как в базовом классе, но вероятность
    /// ошибки растёт пропорционально длине очереди.
    /// </summary>
    protected override FailureKind Evaluate(Packet packet)
    {
        double p = EffectiveProbability(packet) * (1.0 + 0.25 * QueueLength);
        if (p > 1.0)
        {
            p = 1.0;
        }
        return Random.Shared.NextDouble() < p
            ? FailureKind.InvalidData
            : FailureKind.None;
    }

    /// <summary>Учитывает успешно обработанную задачу.</summary>
    protected override void OnProcessingCompleted(NodeEventArgs e)
    {
        _handledCount++;
        base.OnProcessingCompleted(e);
    }

    /// <summary>Рисует узел с пиктограммой шестерёнки.</summary>
    public override void Draw(Graphics g)
    {
        DrawBody(g);
        if (Status != NodeStatus.Failed)
        {
            DrawGearIcon(g);
        }
    }

    // Пиктограмма шестерёнки: диск со спицами и белая ступица.
    private void DrawGearIcon(Graphics g)
    {
        float cx = Location.X;
        float cy = Location.Y + 4f;
        const float r = 11f;
        using (Pen p = new Pen(GearBrush, 3.5f))
        {
            p.StartCap = LineCap.Round;
            p.EndCap = LineCap.Round;
            for (int i = 0; i < 8; i++)
            {
                double angle = i * Math.PI / 4.0;
                float x1 = cx + (float)Math.Cos(angle) * (r - 2f);
                float y1 = cy + (float)Math.Sin(angle) * (r - 2f);
```

```
                    float x2 = cx + (float)Math.Cos(angle) * (r + 3.5f);
                    float y2 = cy + (float)Math.Sin(angle) * (r + 3.5f);
                    g.DrawLine(p, x1, y1, x2, y2);
                }
            }
            g.FillEllipse(GearBrush, cx - r, cy - r, 2f * r, 2f * r);
            g.FillEllipse(HubBrush, cx - 4f, cy - 4f, 8f, 8f);
        }
    }
}
```

**Файл DatabaseNode.cs**

```
using System;
using System.Drawing;

namespace NetworkSim.Model
{
    /// <summary>
    /// База данных — узел, производный от ProcessingNode. Может полностью
    /// отказать (требуется ремонт администратором), причём вероятность отказа
    /// не зависит от номера попытки. Переопределяет Evaluate и Draw.
    /// </summary>
    public class DatabaseNode : ProcessingNode
    {
        private int _outageCount;       // число отказов базы данных

        private static readonly Brush CylinderBrush = new SolidBrush(Color.FromArgb(72, 138, 120));

        /// <summary>Число отказов базы данных (только чтение).</summary>
        public int OutageCount
        {
            get { return _outageCount; }
        }

        /// <summary>Создаёт базу данных с заданными параметрами.</summary>
        public DatabaseNode(string name, string waypointId, PointF location,
                            SimulationClock clock, double processingTime = 2.0,
                            double failureProbability = 0.05)
            : base(name, waypointId, location, clock, processingTime, failureProbability)
        {
        }

        /// <summary>
        /// Определяет исход обработки: с вероятностью FailureProbability
        /// база данных отказывает, иначе пакет обработан успешно.
        /// </summary>
        protected override FailureKind Evaluate(Packet packet)
        {
            if (Random.Shared.NextDouble() < FailureProbability)
            {
                _outageCount++;
                return FailureKind.NodeOutage;
            }
            return FailureKind.None;
        }

        /// <summary>Рисует узел с пиктограммой цилиндра.</summary>
        public override void Draw(Graphics g)
        {
            DrawBody(g);
            if (Status != NodeStatus.Failed)
            {
                DrawCylinderIcon(g);
            }
        }

        // Пиктограмма базы данных: цилиндр.
```

```
        private void DrawCylinderIcon(Graphics g)
        {
            float cx = Location.X;
            float cy = Location.Y + 4f;
            const float w = 24f;
            const float h = 20f;
            const float ry = 5f;
            g.FillEllipse(CylinderBrush, cx - w / 2f, cy - h / 2f - ry, w, 2f * ry);
            g.FillRectangle(CylinderBrush, cx - w / 2f, cy - h / 2f, w, h);
            g.FillEllipse(CylinderBrush, cx - w / 2f, cy + h / 2f - ry, w, 2f * ry);
            using (Pen p = new Pen(Color.White, 1.2f))
            {
                g.DrawEllipse(p, cx - w / 2f, cy - h / 2f - ry, w, 2f * ry);
                g.DrawLine(p, cx - w / 2f, cy - h / 2f + ry, cx - w / 2f, cy + h / 2f);
                g.DrawLine(p, cx + w / 2f, cy - h / 2f + ry, cx + w / 2f, cy + h / 2f);
                g.DrawArc(p, cx - w / 2f, cy + h / 2f - ry, w, 2f * ry, 0f, 180f);
            }
        }
    }
}
```

**Файл NetworkSegment.cs**

```
using System;
using System.Collections.Generic;
using System.Drawing;
using System.Windows.Forms;

namespace NetworkSim.Model
{
    /// <summary>
    /// Базовый класс «окружающей среды» модели — сегмент сети. Владеет графом
    /// дорожек, решает, кому адресовать пакет (ResolveDestination), строит
    /// маршрут и перемещает пакеты; пакеты, адресованные за пределы сегмента,
    /// передаются соседнему сегменту через шлюз. Внешний вид сегмента зависит
    /// от его состояния (State).
    /// </summary>
    public class NetworkSegment : IDrawable
    {
        /// <summary>Часы моделирования, на шаги которых подписан сегмент.</summary>
        protected readonly SimulationClock Clock;

        /// <summary>Граф дорожек сегмента.</summary>
        protected readonly TrackGraph Graph = new TrackGraph();

        private const string GatewayWaypointId = "gw";      // шлюз в графе дорожек

        private readonly List<IPacketEndpoint> _endpoints = new List<IPacketEndpoint>();
        private readonly List<Packet> _inFlight = new List<Packet>();   // пакеты в пути
        private NetworkSegment _neighbor;                   // соседний сегмент
        private PointF _neighborGateway;                    // координата шлюза соседа
        private SegmentState _state = SegmentState.Normal;
        private double _holdRemaining;                      // сколько держится временное состояние,
с

        private static readonly Brush TitleBrush = new SolidBrush(Color.FromArgb(60, 64, 72));
        private static readonly Font TitleFont =
            new Font("Segoe UI", 15f, FontStyle.Bold, GraphicsUnit.Pixel);

        public string Name { get; }

        public RectangleF Bounds { get; }

        /// <summary>Идентификатор вершины-шлюза («выхода» из сегмента).</summary>
        public string GatewayId
        {
            get { return GatewayWaypointId; }
        }
```

```
/// <summary>Состояние сегмента, определяющее его внешний вид.</summary>
public SegmentState State
{
    get { return _state; }
}

/// <summary>Событие: пакет дошёл до шлюза и адресован за пределы сегмента.</summary>
public event EventHandler<PacketEventArgs> PacketLeaving;

/// <summary>Событие: пакет доставлен локальному адресату сегмента.</summary>
public event EventHandler<PacketEventArgs> PacketDelivered;

/// <summary>Событие: пакет отброшен (адресат или маршрут не найдены).</summary>
public event EventHandler<PacketEventArgs> PacketDropped;

/// <summary>Событие: изменилось состояние сегмента.</summary>
public event EventHandler<ValueChangedEventArgs<SegmentState>> StateChanged;

/// <summary>Создаёт сегмент: регистрирует шлюз и подписывается на время.</summary>
public NetworkSegment(string name, RectangleF bounds, PointF gatewayLocation,
                      SimulationClock clock)
{
    Name = name;
    Bounds = bounds;
    Clock = clock;
    Graph.AddWaypoint(GatewayWaypointId, gatewayLocation);
    clock.Tick += OnTick;
}

/// <summary>Регистрирует локального адресата пакетов в сегменте.</summary>
public void AddEndpoint(IPacketEndpoint endpoint)
{
    if (endpoint == null || _endpoints.Contains(endpoint))
    {
        return;
    }
    _endpoints.Add(endpoint);
    Graph.AddWaypoint(endpoint.WaypointId, endpoint.Location);
}

/// <summary>Добавляет вершину графа дорожек сегмента.</summary>
public void AddWaypoint(string id, PointF point)
{
    Graph.AddWaypoint(id, point);
}

/// <summary>Добавляет дорожку между двумя вершинами сегмента.</summary>
public void AddLane(string idA, string idB)
{
    Graph.AddLane(idA, idB);
}

/// <summary>
/// Связывает два сегмента линией связи через шлюзы: пакеты, покидающие
/// один сегмент, передаются на приём другому.
/// </summary>
public void Connect(NetworkSegment other)
{
    if (other == null || _neighbor == other)
    {
        return;
    }
    _neighbor = other;
    _neighborGateway = other.Graph.GetPoint(other.GatewayId);
    other._neighbor = this;
    other._neighborGateway = Graph.GetPoint(GatewayId);
    PacketLeaving += (s, e) => other.Accept(e.Packet);
    other.PacketLeaving += (s, e) => Accept(e.Packet);
```

```
        }

        /// <summary>
        /// Принимает новый пакет, рождённый в этом сегменте (от актора или узла):
        /// определяет адресата, строит маршрут и запускает пакет.
        /// </summary>
        public void Submit(Packet packet)
        {
            if (packet == null)
            {
                return;
            }
            IPacketEndpoint dest = packet.Destination ?? ResolveDestination(packet);
            packet.Destination = dest;

            List<PointF> route;
            if (dest != null && IsLocal(dest))
            {
                route = Graph.FindRoute(packet.SourceWaypointId, dest.WaypointId);
            }
            else
            {
                // Адресат вне сегмента: едем к шлюзу, далее по линии связи.
                route = Graph.FindRoute(packet.SourceWaypointId, GatewayId);
                if (route != null && _neighbor != null)
                {
                    route.Add(_neighborGateway);
                }
            }
            if (route == null)
            {
                OnPacketDropped(new PacketEventArgs(packet));
                return;
            }
            Launch(packet, route);
        }

        /// <summary>Принимает пакет, пришедший через шлюз от соседнего сегмента.</summary>
        public void Accept(Packet packet)
        {
            IPacketEndpoint dest = packet.Destination ?? ResolveDestination(packet);
            packet.Readdress(dest, GatewayId);      // путь продолжается от шлюза
            if (dest == null || !IsLocal(dest))
            {
                OnPacketDropped(new PacketEventArgs(packet));
                return;
            }
            List<PointF> route = Graph.FindRoute(packet.SourceWaypointId, dest.WaypointId);
            if (route == null)
            {
                OnPacketDropped(new PacketEventArgs(packet));
                return;
            }
            Launch(packet, route);
        }

        /// <summary>
        /// Определяет локального адресата пакета по его содержимому;
        /// null означает «адресат вне сегмента». Переопределяется в наследниках.
        /// </summary>
        protected virtual IPacketEndpoint ResolveDestination(Packet packet)
        {
            return packet != null ? packet.Destination : null;
        }

        /// <summary>
        /// Устанавливает состояние сегмента. При holdSeconds &gt; 0 состояние
        /// временное: по истечении этого времени сегмент вернётся к обычному.
        /// </summary>
        protected void SetState(SegmentState state, double holdSeconds)
        {
```

```
            if (holdSeconds > 0)
            {
                _holdRemaining = holdSeconds;
            }
            if (_state != state)
            {
                SegmentState old = _state;
                _state = state;
                OnStateChanged(new ValueChangedEventArgs<SegmentState>(old, state));
            }
        }

        /// <summary>Пересчитывает «постоянное» состояние сегмента.</summary>
        protected void RefreshState()
        {
            SegmentState wanted = ComputeDerivedState();
            if (wanted != SegmentState.Normal)
            {
                _holdRemaining = 0;     // авария и обработка важнее временного «успеха»
            }
            if (_holdRemaining > 0)
            {
                return;
            }
            SetState(wanted, 0);
        }

        /// <summary>Вычисляет «постоянное» состояние по внутренним объектам сегмента.</summary>
        protected virtual SegmentState ComputeDerivedState()
        {
            return SegmentState.Normal;
        }

        /// <summary>Вызывает событие PacketLeaving.</summary>
        protected virtual void OnPacketLeaving(PacketEventArgs e)
        {
            PacketLeaving?.Invoke(this, e);
        }

        /// <summary>Вызывает событие PacketDelivered.</summary>
        protected virtual void OnPacketDelivered(PacketEventArgs e)
        {
            PacketDelivered?.Invoke(this, e);
        }

        /// <summary>Вызывает событие PacketDropped.</summary>
        protected virtual void OnPacketDropped(PacketEventArgs e)
        {
            PacketDropped?.Invoke(this, e);
        }

        /// <summary>Вызывает событие StateChanged.</summary>
        protected virtual void OnStateChanged(ValueChangedEventArgs<SegmentState> e)
        {
            StateChanged?.Invoke(this, e);
        }

        // Является ли объект локальным адресатом этого сегмента.
        private bool IsLocal(IPacketEndpoint endpoint)
        {
            return _endpoints.Contains(endpoint);
        }

        // Запускает пакет по маршруту: подписывается на его прибытие.
        private void Launch(Packet packet, List<PointF> route)
        {
            packet.Arrived += OnPacketArrived;
            _inFlight.Add(packet);
            packet.StartRoute(route);
```

```
        }

        // Пакет дошёл до конца маршрута: доставить адресату или передать соседу.
        private void OnPacketArrived(object sender, PacketEventArgs e)
        {
            Packet packet = e.Packet;
            packet.Arrived -= OnPacketArrived;
            _inFlight.Remove(packet);

            IPacketEndpoint dest = packet.Destination;
            if (dest != null && IsLocal(dest))
            {
                // Сначала фиксируем доставку, потом адресат её обрабатывает.
                OnPacketDelivered(new PacketEventArgs(packet));
                dest.Receive(packet);
            }
            else if (_neighbor != null)
            {
                OnPacketLeaving(new PacketEventArgs(packet));
            }
            else
            {
                OnPacketDropped(new PacketEventArgs(packet));
            }
        }

        // Шаг модельного времени: движение пакетов и отсчёт временного состояния.
        private void OnTick(object sender, TickEventArgs e)
        {
            if (_holdRemaining > 0)
            {
                _holdRemaining -= e.Dt;
                if (_holdRemaining <= 0)
                {
                    _holdRemaining = 0;
                    RefreshState();
                }
            }
            if (_inFlight.Count > 0)
            {
                // Обход по копии списка: обработчик Arrived изменяет список.
                foreach (Packet packet in new List<Packet>(_inFlight))
                {
                    packet.Advance(e.Dt);
                }
            }
        }

        /// <summary>Рисует сегмент: фон по состоянию, рамку, название и дорожки.</summary>
        public virtual void Draw(Graphics g)
        {
            Color fill;
            Color border;
            switch (_state)
            {
                case SegmentState.Busy:
                    fill = Color.FromArgb(255, 247, 214);
                    border = Color.FromArgb(214, 184, 96);
                    break;
                case SegmentState.Alert:
                    fill = Color.FromArgb(253, 227, 227);
                    border = Color.FromArgb(200, 70, 70);
                    break;
                case SegmentState.Success:
                    fill = Color.FromArgb(227, 247, 227);
                    border = Color.FromArgb(96, 175, 96);
                    break;
                default:
                    fill = Color.FromArgb(246, 247, 249);
                    border = Color.FromArgb(150, 158, 168);
                    break;
```

```
            }
            using (SolidBrush b = new SolidBrush(fill))
            {
                g.FillRectangle(b, Bounds.X, Bounds.Y, Bounds.Width, Bounds.Height);
            }
            using (Pen p = new Pen(border, 2f))
            {
                g.DrawRectangle(p, Bounds.X, Bounds.Y, Bounds.Width, Bounds.Height);
            }
            g.DrawString(Name, TitleFont, TitleBrush, Bounds.X + 10f, Bounds.Y + 6f);
            Graph.Draw(g);
        }

        /// <summary>Рисует пакеты, движущиеся по дорожкам сегмента.</summary>
        public void DrawPackets(Graphics g)
        {
            foreach (Packet packet in _inFlight)
            {
                packet.Draw(g);
            }
        }
    }
}
```

**Файл ClientSegment.cs**

```
using System;
using System.Drawing;
using System.Drawing.Drawing2D;

namespace NetworkSim.Model
{
    /// <summary>
    /// Клиентский сегмент сети — производный от NetworkSegment. Объединяет
    /// клиента и пользователя; задачи направляет наружу (в серверный сегмент),
    /// ошибки данных и результаты возвращает клиенту. Пользователь стоит рядом,
    /// видит переданный ему результат и радуется.
    /// Переопределяет ResolveDestination и Draw.
    /// </summary>
    public class ClientSegment : NetworkSegment
    {
        private Client _client;
        private EndUser _endUser;

        private static readonly Pen LinkPen = new Pen(Color.FromArgb(39, 134, 82), 1.8f);

        /// <summary>Создаёт клиентский сегмент и строит его дорожки.</summary>
        public ClientSegment(RectangleF bounds, PointF gatewayLocation, SimulationClock clock)
            : base("Клиентский сегмент", bounds, gatewayLocation, clock)
        {
            // Магистраль сегмента: шлюз — середина — к клиенту.
            AddWaypoint("ct", Layout.ClientTop);
            AddWaypoint("cm", Layout.ClientMiddle);
            AddLane("gw", "cm");
            AddLane("cm", "ct");
        }

        /// <summary>Подключает клиента к сегменту и настраивает реакцию на его события.</summary>
        public void SetClient(Client client)
        {
            if (client == null)
            {
                return;
            }
            _client = client;
            AddWaypoint(client.WaypointId, client.Location);
            AddLane(client.WaypointId, "ct");
            AddEndpoint(client);
            client.PacketSent += OnClientPacketSent;
```

```
            client.FixStarted += OnClientFixStarted;
            client.ResultReceived += OnClientResultReceived;
        }

        /// <summary>Подключает пользователя: он получает результат от клиента.</summary>
        public void SetEndUser(EndUser user)
        {
            if (user == null)
            {
                return;
            }
            _endUser = user;
            user.ResultReceived += OnResultReceived;
        }

        /// <summary>
        /// Определяет адресата пакета: и результат, и ошибка данных возвращаются
        /// клиенту; задачи уходят в серверный сегмент.
        /// </summary>
        protected override IPacketEndpoint ResolveDestination(Packet packet)
        {
            if (packet.Kind == PacketKind.Result || packet.Kind == PacketKind.DataError)
            {
                return _client;
            }
            return null;        // адресат вне сегмента
        }

        // Клиент отправил задачу — запускаем её по дорожкам сегмента.
        private void OnClientPacketSent(object sender, PacketEventArgs e)
        {
            Submit(e.Packet);
        }

        // Клиент начал исправлять ошибку — сегмент переходит в аварийное состояние.
        private void OnClientFixStarted(object sender, EventArgs e)
        {
            SetState(SegmentState.Alert, 2.5);
        }

        // Клиент получил результат — передаёт его пользователю.
        private void OnClientResultReceived(object sender, PacketEventArgs e)
        {
            if (_endUser != null)
            {
                _endUser.ShowResult();
            }
        }

        // Пользователь получил результат — сегмент «празднует» успех.
        private void OnResultReceived(object sender, EventArgs e)
        {
            SetState(SegmentState.Success, 2.0);
        }

        /// <summary>Рисует сегмент и пунктирную связь «клиент — пользователь».</summary>
        public override void Draw(Graphics g)
        {
            base.Draw(g);
            if (_client == null || _endUser == null || _endUser.Mood != UserMood.Happy)
            {
                return;
            }
            LinkPen.DashStyle = DashStyle.Dot;
            LinkPen.EndCap = LineCap.ArrowAnchor;
            g.DrawLine(LinkPen,
                _client.Location.X, _client.Location.Y + 40f,
                _endUser.Location.X, _endUser.Location.Y - 34f);
        }
    }
```

```
}
```

**Файл ServerSegment.cs**

```
using System;
using System.Collections.Generic;
using System.Drawing;

namespace NetworkSim.Model
{
    /// <summary>
    /// Серверный сегмент сети — производный от NetworkSegment. Объединяет
    /// упорядоченную цепочку узлов обработки и администратора; направляет
    /// задачи на первый узел, передаёт пакеты между узлами по цепочке,
    /// уведомления об отказах — администратору, команды ремонта — узлам.
    /// Переопределяет ResolveDestination и ComputeDerivedState.
    /// </summary>
    public class ServerSegment : NetworkSegment
    {
        private readonly List<ProcessingNode> _nodes = new List<ProcessingNode>();
        private Administrator _administrator;

        /// <summary>Создаёт серверный сегмент и строит его дорожки.</summary>
        public ServerSegment(RectangleF bounds, PointF gatewayLocation, SimulationClock clock)
            : base("Серверный сегмент", bounds, gatewayLocation, clock)
        {
            // Магистраль: шлюз → первый узел → … → развилка → последние узлы;
            // от развилки вниз идёт ветка к администратору.
            AddWaypoint("j", Layout.ServerJunction);
            AddLane("gw", "n0");
            AddLane("n0", "n1");
            AddLane("n1", "j");
            AddLane("j", "n2");
            AddLane("n2", "n3");
            AddLane("j", "adm");
        }

        /// <summary>Цепочка узлов обработки (только чтение).</summary>
        public IReadOnlyList<ProcessingNode> Nodes
        {
            get { return _nodes; }
        }

        /// <summary>
        /// Добавляет узел в цепочку и настраивает реакцию сегмента на его
        /// события: завершение, сбой и смену состояния.
        /// </summary>
        public void AddNode(ProcessingNode node)
        {
            if (node == null || _nodes.Contains(node))
            {
                return;
            }
            _nodes.Add(node);
            AddWaypoint(node.WaypointId, node.Location);
            AddEndpoint(node);
            SubscribeToNode(node);
            node.StatusChanged += OnNodeStatusChanged;
        }

        /// <summary>
        /// Подписывает сегмент на события узла. Обе подписки записаны явно,
        /// через new ...Handler, — так видно, что используются собственные
        /// делегаты NodeCompletedHandler и NodeFailedHandler.
        /// </summary>
        private void SubscribeToNode(ProcessingNode node)
        {
            node.ProcessingCompleted += new NodeCompletedHandler(OnNodeCompleted);
            node.ProcessingFailed += new NodeFailedHandler(OnNodeFailed);
```

```
    }

    /// <summary>Подключает администратора к сегменту.</summary>
    public void SetAdministrator(Administrator admin)
    {
        if (admin == null)
        {
            return;
        }
        _administrator = admin;
        AddWaypoint(admin.WaypointId, admin.Location);
        AddEndpoint(admin);
        // Отправленная администратором команда ремонта запускается в сегменте.
        admin.PacketSent += (s, e) => Submit(e.Packet);
    }

    /// <summary>
    /// Определяет адресата пакета: задача — первому узлу цепочки,
    /// уведомление об отказе — администратору, команда ремонта — узлу;
    /// результат уходит в клиентский сегмент.
    /// </summary>
    protected override IPacketEndpoint ResolveDestination(Packet packet)
    {
        switch (packet.Kind)
        {
            case PacketKind.Task:
                return _nodes.Count > 0 ? _nodes[0] : null;
            case PacketKind.NodeFailure:
                return _administrator;
            case PacketKind.Fix:
                return packet.Subject;
            default:
                return null;        // адресат вне сегмента
        }
    }

    /// <summary>Вычисляет состояние сегмента по состояниям узлов.</summary>
    protected override SegmentState ComputeDerivedState()
    {
        bool anyFailed = false;
        bool anyProcessing = false;
        foreach (ProcessingNode node in _nodes)
        {
            if (node.Status == NodeStatus.Failed)
            {
                anyFailed = true;
            }
            else if (node.Status == NodeStatus.Processing)
            {
                anyProcessing = true;
            }
        }
        if (anyFailed)
        {
            return SegmentState.Alert;
        }
        return anyProcessing ? SegmentState.Busy : SegmentState.Normal;
    }

    // Узел завершил обработку: пакет уходит следующему узлу, а после
    // последнего узла формируется результат для клиента.
    private void OnNodeCompleted(object sender, NodeEventArgs e)
    {
        int index = _nodes.IndexOf(e.Node);
        if (index >= 0 && index < _nodes.Count - 1)
        {
            e.Packet.Readdress(_nodes[index + 1], e.Node.WaypointId);
            Submit(e.Packet);
        }
        else
        {
```

```
                Packet result = new Packet(PacketKind.Result,
                    e.Node.WaypointId, e.Packet.Owner, e.Packet.Attempt);
                Submit(result);
                SetState(SegmentState.Success, 1.5);
            }
        }

        // Узел сообщил о сбое: ошибка данных возвращается клиенту,
        // отказ узла — уведомление администратору.
        private void OnNodeFailed(object sender, NodeFailureEventArgs e)
        {
            if (e.Kind == FailureKind.InvalidData)
            {
                Packet error = new Packet(PacketKind.DataError,
                    e.Node.WaypointId, e.Packet.Owner, e.Packet.Attempt);
                error.Destination = e.Packet.Owner;     // адресат вне сегмента
                Submit(error);
            }
            else if (e.Kind == FailureKind.NodeOutage)
            {
                Packet alert = new Packet(PacketKind.NodeFailure,
                    e.Node.WaypointId, e.Packet.Owner, e.Packet.Attempt, e.Node);
                Submit(alert);      // адресата определит ResolveDestination
            }
        }

        // Состояние узла изменилось — пересчитать состояние сегмента.
        private void OnNodeStatusChanged(object sender, ValueChangedEventArgs<NodeStatus> e)
        {
            RefreshState();
        }
    }
}
```

**Файл Client.cs**

```
using System;
using System.Collections.Generic;
using System.Drawing;

namespace NetworkSim.Model
{
    /// <summary>
    /// Клиент — источник задач и получатель результата. Отправляет задачи
    /// в сеть, при получении ошибки данных исправляет её за время FixTime
    /// и отправляет задачу повторно (с начала цепочки узлов); получив
    /// результат, передаёт его пользователю. После MaxAttempts неудачных
    /// попыток задача отменяется.
    /// </summary>
    public class Client : IPacketEndpoint, IDrawable
    {
        protected readonly SimulationClock Clock;

        private const float IconWidth = 52f;
        private const float IconHeight = 36f;
        private const double DoneTime = 2.0;        // сколько показывается «результат получен», с

        private readonly Queue<Packet> _fixQueue = new Queue<Packet>(); // полученные ошибки
        private double _fixRemaining;       // оставшееся время исправления, с
        private double _doneRemaining;      // оставшееся время показа результата, с
        private double _autoSendRemaining;  // до автоматической отправки, с
        private ClientState _state = ClientState.Idle;
        private double _fixTime;
        private int _maxAttempts;
        private double _autoSendInterval;

        private static readonly Brush MonitorBrush = new SolidBrush(Color.FromArgb(70, 90, 120));
        private static readonly Brush ScreenBrush = new SolidBrush(Color.FromArgb(235, 241, 248));
```

```
        private static readonly Brush ScreenAlertBrush = new SolidBrush(Color.FromArgb(250, 205,
205));
        private static readonly Brush ScreenDoneBrush = new SolidBrush(Color.FromArgb(205, 240,
210));
        private static readonly Brush ProgressBrush = new SolidBrush(Color.FromArgb(192, 57, 43));
        private static readonly Brush DoneTextBrush = new SolidBrush(Color.FromArgb(39, 134, 82));
        private static readonly Brush LabelBrush = new SolidBrush(Color.FromArgb(40, 44, 52));
        private static readonly Pen MonitorPen = new Pen(Color.FromArgb(40, 44, 52), 1.4f);
        private static readonly Font LabelFont = new Font("Segoe UI", 12f, FontStyle.Bold,
GraphicsUnit.Pixel);
        private static readonly Font HintFont = new Font("Segoe UI", 11f, FontStyle.Regular,
GraphicsUnit.Pixel);

        public string Name { get; }

        public string WaypointId { get; }

        public PointF Location { get; }

        /// <summary>Состояние клиента (снаружи только чтение).</summary>
        public ClientState State
        {
            get { return _state; }
        }

        /// <summary>Время исправления ошибки данных, с (должно быть больше нуля).</summary>
        public double FixTime
        {
            get { return _fixTime; }
            set
            {
                if (value <= 0)
                {
                    throw new ArgumentOutOfRangeException(nameof(value),
                        "Время исправления ошибки должно быть положительным числом.");
                }
                _fixTime = value;
            }
        }

        /// <summary>Максимальное число попыток выполнения задачи (не меньше 1).</summary>
        public int MaxAttempts
        {
            get { return _maxAttempts; }
            set
            {
                if (value < 1)
                {
                    throw new ArgumentOutOfRangeException(nameof(value),
                        "Число попыток должно быть не меньше 1.");
                }
                _maxAttempts = value;
            }
        }

        /// <summary>Автоматическая отправка задач через равные интервалы.</summary>
        public bool AutoSend { get; set; }

        /// <summary>Период автоматической отправки задач, с (больше нуля).</summary>
        public double AutoSendInterval
        {
            get { return _autoSendInterval; }
            set
            {
                if (value <= 0)
                {
                    throw new ArgumentOutOfRangeException(nameof(value),
                        "Период автоотправки должен быть положительным числом.");
                }
```

```
            _autoSendInterval = value;
        }
    }

    /// <summary>Число отправленных задач (с учётом повторов, только чтение).</summary>
    public int SentCount { get; private set; }

    /// <summary>
    /// Внешнее условие возможности отправки задачи; задаётся главной
    /// программой (ограничение числа задач «в полёте»).
    /// </summary>
    public Func<bool> SendGate { get; set; }

    /// <summary>Событие: клиент отправил задачу.</summary>
    public event EventHandler<PacketEventArgs> PacketSent;

    /// <summary>Событие: клиент начал исправлять ошибку данных.</summary>
    public event EventHandler FixStarted;

    /// <summary>Событие: попытки исчерпаны, задача отменена.</summary>
    public event EventHandler TaskAbandoned;

    /// <summary>Событие: клиент получил результат задачи.</summary>
    public event EventHandler<PacketEventArgs> ResultReceived;

    /// <summary>Создаёт клиента и подписывает его на шаги модельного времени.</summary>
    public Client(string name, string waypointId, PointF location, SimulationClock clock)
    {
        Name = name;
        WaypointId = waypointId;
        Location = location;
        Clock = clock;
        FixTime = 2.0;
        MaxAttempts = 3;
        AutoSendInterval = 6.0;
        clock.Tick += OnTick;
    }

    /// <summary>Создаёт и отправляет новую задачу (попытка 1).</summary>
    public void SendTask()
    {
        if (SendGate != null && !SendGate())
        {
            return;     // главная программа запретила отправку
        }
        Packet packet = new Packet(PacketKind.Task, WaypointId, this, 1);
        SentCount++;
        OnPacketSent(new PacketEventArgs(packet));
    }

    /// <summary>
    /// Принимает пакет: результат задачи или сообщение об ошибке данных.
    /// Ошибки встают в очередь на исправление.
    /// </summary>
    public void Receive(Packet packet)
    {
        if (packet == null)
        {
            return;
        }
        if (packet.Kind == PacketKind.DataError)
        {
            _fixQueue.Enqueue(packet);
            if (_state != ClientState.Fixing)
            {
                StartFixing();
            }
        }
        else if (packet.Kind == PacketKind.Result)
```

```
            {
                _state = ClientState.Done;
                _doneRemaining = DoneTime;
                OnResultReceived(new PacketEventArgs(packet));
            }
        }

        /// <summary>Вызывает событие PacketSent.</summary>
        protected virtual void OnPacketSent(PacketEventArgs e)
        {
            PacketSent?.Invoke(this, e);
        }

        /// <summary>Вызывает событие FixStarted.</summary>
        protected virtual void OnFixStarted(EventArgs e)
        {
            FixStarted?.Invoke(this, e);
        }

        /// <summary>Вызывает событие TaskAbandoned.</summary>
        protected virtual void OnTaskAbandoned(EventArgs e)
        {
            TaskAbandoned?.Invoke(this, e);
        }

        /// <summary>Вызывает событие ResultReceived.</summary>
        protected virtual void OnResultReceived(PacketEventArgs e)
        {
            ResultReceived?.Invoke(this, e);
        }

        // Начинает исправление очередной ошибки данных.
        private void StartFixing()
        {
            _state = ClientState.Fixing;
            _fixRemaining = FixTime;
            OnFixStarted(EventArgs.Empty);
        }

        // Исправление завершено: повторная отправка задачи или отмена.
        private void ResendAfterFix()
        {
            Packet failed = _fixQueue.Dequeue();
            int nextAttempt = failed.Attempt + 1;
            if (nextAttempt > MaxAttempts)
            {
                OnTaskAbandoned(EventArgs.Empty);
                return;
            }
            Packet retry = new Packet(PacketKind.Task, WaypointId, this, nextAttempt);
            SentCount++;
            OnPacketSent(new PacketEventArgs(retry));
        }

        // Шаг модельного времени: исправление, показ результата, автоотправка.
        private void OnTick(object sender, TickEventArgs e)
        {
            if (_state == ClientState.Fixing)
            {
                _fixRemaining -= e.Dt;
                if (_fixRemaining <= 0)
                {
                    _state = ClientState.Idle;
                    ResendAfterFix();
                    if (_state == ClientState.Idle && _fixQueue.Count > 0)
                    {
                        StartFixing();      // в очереди есть ещё ошибки
                    }
                }
            }
```

```
            else if (_state == ClientState.Done)
            {
                _doneRemaining -= e.Dt;
                if (_doneRemaining <= 0)
                {
                    _state = ClientState.Idle;
                }
            }

            if (AutoSend)
            {
                _autoSendRemaining -= e.Dt;
                if (_autoSendRemaining <= 0)
                {
                    _autoSendRemaining = AutoSendInterval;
                    SendTask();
                }
            }
        }

        /// <summary>Рисует клиента: монитор, подпись и состояние обмена с сетью.</summary>
        public void Draw(Graphics g)
        {
            RectangleF screen = new RectangleF(Location.X - IconWidth / 2f,
                Location.Y - IconHeight / 2f - 4f, IconWidth, IconHeight);

            Brush screenBrush = ScreenBrush;
            if (_state == ClientState.Fixing) screenBrush = ScreenAlertBrush;
            else if (_state == ClientState.Done) screenBrush = ScreenDoneBrush;

            g.FillRectangle(screenBrush, screen);
            g.DrawRectangle(MonitorPen, screen.X, screen.Y, screen.Width, screen.Height);
            g.FillRectangle(MonitorBrush, Location.X - 5f, Location.Y + IconHeight / 2f - 4f, 10f,
6f);
            g.FillRectangle(MonitorBrush, Location.X - 16f, Location.Y + IconHeight / 2f + 2f, 32f,
4f);

            if (_state == ClientState.Fixing)
            {
                // Полоса прогресса исправления ошибки.
                double fraction = 1.0 - Math.Max(0, _fixRemaining) / FixTime;
                if (fraction < 0) fraction = 0;
                if (fraction > 1) fraction = 1;
                RectangleF bar = new RectangleF(screen.X + 4f, screen.Bottom - 8f, screen.Width -
8f, 5f);
                g.FillRectangle(ScreenBrush, bar);
                g.FillRectangle(ProgressBrush, bar.X, bar.Y, (float)(bar.Width * fraction),
bar.Height);
            }

            g.DrawString(Name, LabelFont, LabelBrush, Location.X - 22f, Location.Y + 28f);
            if (_state == ClientState.Fixing)
            {
                g.DrawString("исправляет ошибку…", HintFont, ProgressBrush,
                    Location.X - 50f, Location.Y + 46f);
            }
            else if (_state == ClientState.Done)
            {
                g.DrawString("результат получен", HintFont, DoneTextBrush,
                    Location.X - 50f, Location.Y + 46f);
            }
        }
    }
}
```

**Файл Administrator.cs**

```
using System;
using System.Collections.Generic;
```

```
using System.Drawing;
using System.Drawing.Drawing2D;

namespace NetworkSim.Model
{
    /// <summary>
    /// Администратор — специалист, обслуживающий серверный сегмент. Получает
    /// уведомления об отказах узлов, «подключается по SSH» к отказавшему узлу
    /// и по истечении времени ремонта отправляет ему команду ремонта (Fix).
    /// </summary>
    public class Administrator : IPacketEndpoint, IDrawable
    {
        protected readonly SimulationClock Clock;

        private readonly Queue<Packet> _requests = new Queue<Packet>(); // уведомления об отказах
        private double _repairRemaining;    // оставшееся время ремонта, с
        private AdminState _state = AdminState.Idle;
        private double _repairTime;

        private static readonly Brush PersonBrush = new SolidBrush(Color.FromArgb(52, 73, 110));
        private static readonly Brush LabelBrush = new SolidBrush(Color.FromArgb(40, 44, 52));
        private static readonly Brush HintBrush = new SolidBrush(Color.FromArgb(46, 116, 181));
        private static readonly Font LabelFont = new Font("Segoe UI", 12f, FontStyle.Bold,
GraphicsUnit.Pixel);
        private static readonly Font HintFont = new Font("Segoe UI", 11f, FontStyle.Regular,
GraphicsUnit.Pixel);

        public string Name { get; }

        public string WaypointId { get; }

        public PointF Location { get; }

        /// <summary>Состояние администратора (снаружи только чтение).</summary>
        public AdminState State
        {
            get { return _state; }
        }

        /// <summary>Время ремонта одного узла, с (должно быть больше нуля).</summary>
        public double RepairTime
        {
            get { return _repairTime; }
            set
            {
                if (value <= 0)
                {
                    throw new ArgumentOutOfRangeException(nameof(value),
                        "Время ремонта должно быть положительным числом.");
                }
                _repairTime = value;
            }
        }

        /// <summary>Число выполненных ремонтов (только чтение).</summary>
        public int RepairsDone { get; private set; }

        /// <summary>Узел, к которому администратор подключён в данный момент (или null).</summary>
        public ProcessingNode CurrentTarget { get; private set; }

        /// <summary>Событие: администратор отправил команду ремонта узлу.</summary>
        public event EventHandler<PacketEventArgs> PacketSent;

        /// <summary>Событие: администратор начал ремонт (подключился по SSH).</summary>
        public event EventHandler RepairStarted;

        /// <summary>Создаёт администратора и подписывает его на шаги времени.</summary>
```

```
        public Administrator(string name, string waypointId, PointF location, SimulationClock clock)
        {
            Name = name;
            WaypointId = waypointId;
            Location = location;
            Clock = clock;
            RepairTime = 3.0;
            clock.Tick += OnTick;
        }

        /// <summary>Принимает пакет: ожидается уведомление об отказе узла.</summary>
        public void Receive(Packet packet)
        {
            if (packet == null || packet.Kind != PacketKind.NodeFailure)
            {
                return;
            }
            _requests.Enqueue(packet);
            if (_state == AdminState.Idle)
            {
                StartRepair();
            }
        }

        /// <summary>Вызывает событие PacketSent.</summary>
        protected virtual void OnPacketSent(PacketEventArgs e)
        {
            PacketSent?.Invoke(this, e);
        }

        /// <summary>Вызывает событие RepairStarted.</summary>
        protected virtual void OnRepairStarted(EventArgs e)
        {
            RepairStarted?.Invoke(this, e);
        }

        // Начинает ремонт очередного отказавшего узла.
        private void StartRepair()
        {
            Packet request = _requests.Dequeue();
            _state = AdminState.Repairing;
            CurrentTarget = request.Subject;
            _repairRemaining = RepairTime;
            OnRepairStarted(EventArgs.Empty);
        }

        // Ремонт завершён: отправить узлу команду ремонта (пакет Fix).
        private void FinishRepair()
        {
            ProcessingNode target = CurrentTarget;
            _state = AdminState.Idle;
            CurrentTarget = null;
            RepairsDone++;
            if (target != null)
            {
                Packet fix = new Packet(PacketKind.Fix, WaypointId, this, 1, target);
                fix.Destination = target;
                OnPacketSent(new PacketEventArgs(fix));
            }
            if (_requests.Count > 0)
            {
                StartRepair();      // в очереди есть ещё отказы
            }
        }

        // Шаг модельного времени: отсчёт времени ремонта.
        private void OnTick(object sender, TickEventArgs e)
        {
            if (_state != AdminState.Repairing)
            {
```

```
                return;
            }
            _repairRemaining -= e.Dt;
            if (_repairRemaining <= 0)
            {
                FinishRepair();
            }
        }

        /// <summary>
        /// Рисует администратора; во время ремонта — пунктирную анимированную
        /// линию «SSH» к ремонтируемому узлу.
        /// </summary>
        public void Draw(Graphics g)
        {
            if (_state == AdminState.Repairing && CurrentTarget != null)
            {
                PointF target = CurrentTarget.Location;
                using (Pen p = new Pen(HintBrush, 1.8f))
                {
                    p.DashStyle = DashStyle.Dash;
                    p.DashOffset = -(float)(Clock.Elapsed * 12.0);   // «бегущий» пунктир
                    g.DrawLine(p, Location.X, Location.Y - 14f,
                        target.X, target.Y + ProcessingNode.NodeHeight / 2f);
                }
                g.DrawString("SSH", HintFont, HintBrush,
                    (Location.X + target.X) / 2f + 6f,
                    (Location.Y - 14f + target.Y + ProcessingNode.NodeHeight / 2f) / 2f);
            }

            // Фигура человека: голова и плечи.
            g.FillEllipse(PersonBrush, Location.X - 8f, Location.Y - 24f, 16f, 16f);
            g.FillEllipse(PersonBrush, Location.X - 18f, Location.Y - 4f, 36f, 26f);
            g.DrawString(Name, LabelFont, LabelBrush, Location.X - 50f, Location.Y + 27f);
            if (_state == AdminState.Repairing)
            {
                g.DrawString("ремонтирует…", HintFont, HintBrush,
                    Location.X - 32f, Location.Y + 45f);
            }
        }
    }
}
```

**Файл EndUser.cs**

```
using System;
using System.Drawing;

namespace NetworkSim.Model
{
    /// <summary>
    /// Получатель результата (конечный пользователь). Стоит рядом с клиентом
    /// и радуется, когда клиент передаёт ему готовый результат. Пакеты
    /// пользователю не адресуются: результат всегда возвращается клиенту.
    /// </summary>
    public class EndUser : IDrawable
    {
        protected readonly SimulationClock Clock;

        private const float HappyTime = 2.0f;   // длительность радости, с

        private double _happyRemaining;         // сколько ещё длится радость, с
        private UserMood _mood = UserMood.Waiting;

        private static readonly Brush PersonBrush = new SolidBrush(Color.FromArgb(126, 96, 60));
        private static readonly Brush LabelBrush = new SolidBrush(Color.FromArgb(40, 44, 52));
        private static readonly Brush HappyBrush = new SolidBrush(Color.FromArgb(39, 134, 82));
        private static readonly Pen FacePen = new Pen(Color.FromArgb(40, 44, 52), 1.4f);
```

```
        private static readonly Font LabelFont = new Font("Segoe UI", 12f, FontStyle.Bold,
GraphicsUnit.Pixel);
        private static readonly Font HintFont = new Font("Segoe UI", 11f, FontStyle.Regular,
GraphicsUnit.Pixel);

        public string Name { get; }

        public PointF Location { get; }

        /// <summary>Настроение: ожидание или довольный (снаружи только чтение).</summary>
        public UserMood Mood
        {
            get { return _mood; }
        }

        /// <summary>Число полученных результатов (только чтение).</summary>
        public int ResultsReceived { get; private set; }

        /// <summary>Событие: результат задачи получен.</summary>
        public event EventHandler ResultReceived;

        /// <summary>Создаёт получателя результата и подписывает его на шаги времени.</summary>
        public EndUser(string name, PointF location, SimulationClock clock)
        {
            Name = name;
            Location = location;
            Clock = clock;
            clock.Tick += OnTick;
        }

        /// <summary>Клиент передал пользователю готовый результат — тот радуется.</summary>
        public void ShowResult()
        {
            ResultsReceived++;
            _mood = UserMood.Happy;
            _happyRemaining = HappyTime;
            OnResultReceived(EventArgs.Empty);
        }

        /// <summary>Вызывает событие ResultReceived.</summary>
        protected virtual void OnResultReceived(EventArgs e)
        {
            ResultReceived?.Invoke(this, e);
        }

        // Шаг модельного времени: отсчёт длительности радости.
        private void OnTick(object sender, TickEventArgs e)
        {
            if (_mood != UserMood.Happy)
            {
                return;
            }
            _happyRemaining -= e.Dt;
            if (_happyRemaining <= 0)
            {
                _mood = UserMood.Waiting;
            }
        }

        /// <summary>Рисует пользователя: фигуру человека и выражение лица.</summary>
        public void Draw(Graphics g)
        {
            g.FillEllipse(PersonBrush, Location.X - 9f, Location.Y - 26f, 18f, 18f);
            g.FillEllipse(PersonBrush, Location.X - 19f, Location.Y - 4f, 38f, 27f);
            if (_mood == UserMood.Happy)
            {
                // Улыбка на «лице» (голова — круг выше центра Location).
                g.DrawArc(FacePen, Location.X - 7f, Location.Y - 19f, 14f, 10f, 20f, 140f);
            }
```

```
            else
            {
                g.DrawLine(FacePen, Location.X - 5f, Location.Y - 13f,
                    Location.X + 5f, Location.Y - 13f);
            }
            g.DrawString(Name, LabelFont, LabelBrush, Location.X - 46f, Location.Y + 28f);
            if (_mood == UserMood.Happy)
            {
                g.DrawString("результат получен!", HintFont, HappyBrush,
                    Location.X - 52f, Location.Y + 46f);
            }
        }
    }
}
```

**Файл SimulationLogger.cs**

```
using System;

namespace NetworkSim.Model
{
    /// <summary>Аргументы события записи в журнал моделирования.</summary>
    public class LogEventArgs : EventArgs
    {
        public double Time { get; }

        public string Source { get; }

        public string Message { get; }

        public LogEventArgs(double time, string source, string message)
        {
            Time = time;
            Source = source;
            Message = message;
        }

        /// <summary>Готовая строка для отображения в журнале.</summary>
        public string Line
        {
            get { return string.Format("[{0:0.0} с] {1}: {2}", Time, Source, Message); }
        }
    }

    /// <summary>
    /// Журнал моделирования. Объекты модели не зависят от интерфейса: они
    /// сообщают о своих действиях событиями, а журнал (наблюдатель) слушает
    /// эти события и уведомляет главную форму о новых записях.
    /// </summary>
    public class SimulationLogger
    {
        private readonly SimulationClock _clock;    // для отметки модельного времени

        /// <summary>Событие: добавлена новая запись журнала.</summary>
        public event EventHandler<LogEventArgs> MessageLogged;

        /// <summary>Создаёт журнал, привязанный к часам моделирования.</summary>
        public SimulationLogger(SimulationClock clock)
        {
            _clock = clock;
        }

        /// <summary>Добавляет запись в журнал с текущим модельным временем.</summary>
        public void Log(string source, string message)
        {
            OnMessageLogged(new LogEventArgs(_clock.Elapsed, source, message));
        }
```

```
        /// <summary>Вызывает событие MessageLogged.</summary>
        protected virtual void OnMessageLogged(LogEventArgs e)
        {
            MessageLogged?.Invoke(this, e);
        }
    }
}
```

**Файл SimulationWorld.cs**

```
using System;
using System.Collections.Generic;
using System.Drawing;

namespace NetworkSim.Model
{
    /// <summary>
    /// «Сборщик» модели: создаёт часы, сегменты сети, узлы, акторов и журнал,
    /// связывает объекты подписками на события и ведёт общие счётчики.
    /// Главная форма работает с моделью только через этот класс.
    /// </summary>
    public class SimulationWorld : IDrawable
    {
        private readonly List<ProcessingNode> _nodes = new List<ProcessingNode>();

        private int _errorsReturned;        // ошибок данных возвращено клиенту
        private int _tasksActive;           // задач «в полёте»

        /// <summary>Максимальное число задач, одновременно находящихся в работе.</summary>
        public const int MaxActiveTasks = 5;

        public SimulationClock Clock { get; }

        public ClientSegment ClientSide { get; }

        public ServerSegment ServerSide { get; }

        public Client Client { get; }

        public EndUser EndUser { get; }

        public Administrator Administrator { get; }

        public SimulationLogger Logger { get; }

        /// <summary>Список узлов обработки серверного сегмента (только чтение).</summary>
        public IReadOnlyList<ProcessingNode> Nodes
        {
            get { return _nodes; }
        }

        /// <summary>Всего отправлено задач (с учётом повторов).</summary>
        public int SentCount
        {
            get { return Client.SentCount; }
        }

        /// <summary>Доставлено результатов пользователю.</summary>
        public int DeliveredCount
        {
            get { return EndUser.ResultsReceived; }
        }

        /// <summary>Ошибок данных возвращено клиенту.</summary>
```

```
        public int ErrorsReturned
        {
            get { return _errorsReturned; }
        }

        /// <summary>Выполнено ремонтов узлов.</summary>
        public int RepairsDone
        {
            get { return Administrator.RepairsDone; }
        }

        /// <summary>Задач сейчас «в полёте» (принято к работе, но не завершено).</summary>
        public int TasksActive
        {
            get { return _tasksActive; }
        }



        /// <summary>
        /// Создаёт и связывает все объекты модели: часы, журнал, два сегмента,
        /// четыре узла (включая экземпляр базового класса — коммутатор) и акторов.
        /// </summary>
        public SimulationWorld()
        {
            Clock = new SimulationClock();
            Logger = new SimulationLogger(Clock);

            ClientSide = new ClientSegment(Layout.ClientSegmentBounds,
                Layout.ClientGateway, Clock);
            ServerSide = new ServerSegment(Layout.ServerSegmentBounds,
                Layout.ServerGateway, Clock);

            Client = new Client("Клиент", Layout.ClientWaypointId,
                Layout.ClientLocation, Clock);
            // Подписка раньше сегмента: в журнале сначала «клиент получил»,
            // затем реакция пользователя.
            Client.ResultReceived += (s, e) => Logger.Log(Client.Name,
                "результат получен и передан пользователю");
            EndUser = new EndUser("Пользователь", Layout.EndUserLocation, Clock);
            EndUser.ResultReceived += (s, e) => Logger.Log(EndUser.Name,
                "результат получен, пользователь доволен");
            Administrator = new Administrator("Администратор", Layout.AdministratorWaypointId,
                Layout.AdministratorLocation, Clock);

            // Цепочка обработки: экземпляр базового класса (коммутатор) и три наследника.
            ProcessingNode switchNode = new ProcessingNode("Коммутатор", "n0",
                Layout.NodeLocation(0), Clock, 1.0, 0.0);
            FirewallNode firewall = new FirewallNode("Брандмауэр", "n1",
                Layout.NodeLocation(1), Clock, 1.5, 0.1);
            AppServerNode appServer = new AppServerNode("Сервер приложений", "n2",
                Layout.NodeLocation(2), Clock, 2.0, 0.1);
            DatabaseNode database = new DatabaseNode("База данных", "n3",
                Layout.NodeLocation(3), Clock, 2.0, 0.05);
            _nodes.Add(switchNode);
            _nodes.Add(firewall);
            _nodes.Add(appServer);
            _nodes.Add(database);

            ClientSide.SetClient(Client);
            ClientSide.SetEndUser(EndUser);
            ServerSide.AddNode(switchNode);
            ServerSide.AddNode(firewall);
            ServerSide.AddNode(appServer);
            ServerSide.AddNode(database);
            ServerSide.SetAdministrator(Administrator);
            ClientSide.Connect(ServerSide);

            // Условие отправки: не больше MaxActiveTasks задач одновременно.
            Client.SendGate = () => _tasksActive < MaxActiveTasks;
```

```
        // Общие счётчики ведутся по событиям объектов.
        Client.PacketSent += (s, e) =>
        {
            if (e.Packet.Attempt == 1)
            {
                _tasksActive++;
            }
        };
        Client.FixStarted += (s, e) => _errorsReturned++;
        Client.TaskAbandoned += (s, e) => _tasksActive--;
        EndUser.ResultReceived += (s, e) => _tasksActive--;
        ClientSide.PacketDropped += OnPacketDropped;
        ServerSide.PacketDropped += OnPacketDropped;

        WireLogger();
    }

    /// <summary>
    /// Применяет настройки модели: вероятность ошибки данных (брандмауэр
    /// и сервер приложений), вероятность отказа (база данных) и время
    /// обработки на всех узлах.
    /// </summary>
    public void ApplySettings(double dataErrorProbability, double outageProbability,
                              double processingTime)
    {
        foreach (ProcessingNode node in _nodes)
        {
            node.ProcessingTime = processingTime;
            FirewallNode firewall = node as FirewallNode;
            if (firewall != null)
            {
                firewall.FailureProbability = dataErrorProbability;
                continue;
            }
            AppServerNode appServer = node as AppServerNode;
            if (appServer != null)
            {
                appServer.FailureProbability = dataErrorProbability;
                continue;
            }
            DatabaseNode database = node as DatabaseNode;
            if (database != null)
            {
                database.FailureProbability = outageProbability;
            }
        }
    }

    /// <summary>
    /// Отправляет новую задачу по команде пользователя. Возвращает false
    /// и пояснение в reason, если моделирование остановлено или достигнут
    /// предел числа задач «в полёте».
    /// </summary>
    public bool TrySendTask(out string reason)
    {
        reason = null;
        if (!Clock.IsRunning)
        {
            reason = "Моделирование остановлено — нажмите «Старт»";
            return false;
        }
        if (_tasksActive >= MaxActiveTasks)
        {
            reason = "Достигнут предел числа задач в полёте (" + MaxActiveTasks + ")";
            return false;
        }
        Client.SendTask();
        return true;
    }
```

```
        /// <summary>Рисует всю сцену: линию связи, сегменты, узлы, акторов и пакеты.</summary>
        public void Draw(Graphics g)
        {
            // Линия связи между шлюзами сегментов.
            using (Pen casing = new Pen(TrackGraph.CasingColor, 9f))
            using (Pen road = new Pen(TrackGraph.RoadColor, 6f))
            {
                casing.StartCap = System.Drawing.Drawing2D.LineCap.Round;
                casing.EndCap = System.Drawing.Drawing2D.LineCap.Round;
                road.StartCap = System.Drawing.Drawing2D.LineCap.Round;
                road.EndCap = System.Drawing.Drawing2D.LineCap.Round;
                g.DrawLine(casing, Layout.ClientGateway, Layout.ServerGateway);
                g.DrawLine(road, Layout.ClientGateway, Layout.ServerGateway);
            }

            ClientSide.Draw(g);
            ServerSide.Draw(g);
            // Пакеты рисуются до узлов: заходя в узел, пакет скрывается за его рамкой.
            ClientSide.DrawPackets(g);
            ServerSide.DrawPackets(g);
            foreach (ProcessingNode node in _nodes)
            {
                node.Draw(g);
            }
            Client.Draw(g);
            EndUser.Draw(g);
            Administrator.Draw(g);
        }

        // Пакет потерян: задача завершилась, счётчик «в полёте» уменьшается.
        private void OnPacketDropped(object sender, PacketEventArgs e)
        {
            if (e.Packet.Kind == PacketKind.Task && _tasksActive > 0)
            {
                _tasksActive--;
            }
        }

        // Подписывает журнал на события всех объектов: объекты не знают
        // друг о друге, наблюдатель просто слушает их события.
        private void WireLogger()
        {
            Client.PacketSent += (s, e) => Logger.Log(Client.Name,
                "задача отправлена (пакет #" + e.Packet.Id + ", попытка " + e.Packet.Attempt + ")");
            Client.FixStarted += (s, e) => Logger.Log(Client.Name, "исправление ошибки данных…");
            Client.TaskAbandoned += (s, e) => Logger.Log(Client.Name, "попытки исчерпаны, задача
отменена");

            Administrator.RepairStarted += (s, e) => Logger.Log(Administrator.Name,
                "подключение по SSH к узлу «" + Administrator.CurrentTarget.Name + "»");
            Administrator.PacketSent += (s, e) => Logger.Log(Administrator.Name,
                "отправлена команда ремонта узлу «" + e.Packet.Subject.Name + "»");



            ClientSide.StateChanged += (s, e) => Logger.Log(ClientSide.Name,
                "состояние сегмента: " + SegmentStateText(e.NewValue));
            ServerSide.StateChanged += (s, e) => Logger.Log(ServerSide.Name,
                "состояние сегмента: " + SegmentStateText(e.NewValue));
            ClientSide.PacketLeaving += (s, e) => Logger.Log(ClientSide.Name,
                "пакет #" + e.Packet.Id + " отправлен в серверный сегмент");
            ServerSide.PacketLeaving += (s, e) => Logger.Log(ServerSide.Name,
                "пакет #" + e.Packet.Id + " отправлен в клиентский сегмент");
            ClientSide.PacketDelivered += (s, e) => Logger.Log(ClientSide.Name,
                "пакет #" + e.Packet.Id + " доставлен адресату «" + e.Packet.Destination.Name +
"»");
            ServerSide.PacketDelivered += (s, e) => Logger.Log(ServerSide.Name,
                "пакет #" + e.Packet.Id + " доставлен адресату «" + e.Packet.Destination.Name +
"»");
            ClientSide.PacketDropped += (s, e) => Logger.Log(ClientSide.Name,
```

```
                "пакет #" + e.Packet.Id + " потерян (маршрут не найден)");
            ServerSide.PacketDropped += (s, e) => Logger.Log(ServerSide.Name,
                "пакет #" + e.Packet.Id + " потерян (маршрут не найден)");

            foreach (ProcessingNode node in _nodes)
            {
                node.ProcessingCompleted += (s, e) =>
                {
                    string text = "пакет #" + e.Packet.Id + " (попытка " +
                                  e.Packet.Attempt + ") обработан";
                    AppServerNode appServer = e.Node as AppServerNode;
                    if (appServer != null)
                    {
                        text += " (всего обработано: " + appServer.HandledCount + ")";
                    }
                    Logger.Log(e.Node.Name, text);
                };
                node.ProcessingFailed += (s, e) =>
                {
                    if (e.Kind == FailureKind.InvalidData)
                    {
                        string text = "ОШИБКА ДАННЫХ в пакете #" + e.Packet.Id +
                                      " (попытка " + e.Packet.Attempt + "), пакет возвращён
клиенту";
                        FirewallNode firewall = e.Node as FirewallNode;
                        if (firewall != null)
                        {
                            text += " (всего блокировок: " + firewall.BlockedCount + ")";
                        }
                        Logger.Log(e.Node.Name, text);
                    }
                    else
                    {
                        string text = "ОТКАЗ УЗЛА: пакет #" + e.Packet.Id +
                                      " удерживается до ремонта";
                        DatabaseNode database = e.Node as DatabaseNode;
                        if (database != null)
                        {
                            text += " (всего отказов: " + database.OutageCount + ")";
                        }
                        Logger.Log(e.Node.Name, text);
                    }
                };
                node.StatusChanged += (s, e) => Logger.Log(node.Name,
                    "состояние: " + NodeStatusText(e.NewValue));
            }
        }

        // Текстовое описание состояния узла для журнала.
        private static string NodeStatusText(NodeStatus status)
        {
            switch (status)
            {
                case NodeStatus.Processing: return "обработка";
                case NodeStatus.Failed: return "ОТКАЗ";
                default: return "простой";
            }
        }

        // Текстовое описание состояния сегмента для журнала.
        private static string SegmentStateText(SegmentState state)
        {
            switch (state)
            {
                case SegmentState.Busy: return "обработка";
                case SegmentState.Alert: return "авария";
                case SegmentState.Success: return "успешно";
                default: return "штатный режим";
            }
        }
    }
```

```
}
```

**Файл Layout.cs**

```
using System.Drawing;

namespace NetworkSim.Model
{
    /// <summary>
    /// Статический класс с координатами всех объектов сцены: сегментов,
    /// шлюзов, узлов, акторов и точек магистралей. Собран в одном месте,
    /// чтобы схему можно было править в одном месте.
    /// </summary>
    public static class Layout
    {
        /// <summary>Ширина логического холста схемы.</summary>
        public const int CanvasWidth = 1060;

        /// <summary>Высота логического холста схемы.</summary>
        public const int CanvasHeight = 540;

        // Границы сегментов сети на холсте.
        public static readonly RectangleF ClientSegmentBounds = new RectangleF(20f, 20f, 300f,
500f);
        public static readonly RectangleF ServerSegmentBounds = new RectangleF(380f, 20f, 660f,
500f);

        // Шлюзы сегментов (между ними проходит линия связи).
        public static readonly PointF ClientGateway = new PointF(320f, 270f);
        public static readonly PointF ServerGateway = new PointF(380f, 270f);

        // Акторы клиентского и серверного сегментов.
        public static readonly PointF ClientLocation = new PointF(110f, 130f);
        public static readonly PointF EndUserLocation = new PointF(110f, 330f);
        public static readonly PointF AdministratorLocation = new PointF(685f, 450f);

        // Идентификаторы вершин акторов в графах дорожек.
        public const string ClientWaypointId = "cl";
        public const string AdministratorWaypointId = "adm";

        // Промежуточные точки магистрали клиентского сегмента.
        public static readonly PointF ClientTop = new PointF(200f, 190f);
        public static readonly PointF ClientMiddle = new PointF(200f, 270f);

        // Развилка магистрали серверного сегмента (от неё идёт ветка к администратору).
        public static readonly PointF ServerJunction = new PointF(685f, 270f);

        // Центры узлов обработки: четыре узла на одной магистрали.
        public static PointF NodeLocation(int nodeIndex)
        {
            return new PointF(460f + nodeIndex * 150f, 270f);
        }
    }
}
```

**Файл SimulationCanvas.cs**

```
using System;
using System.Drawing;
using System.Drawing.Drawing2D;
using System.Drawing.Text;
```

```
using System.Windows.Forms;
using NetworkSim.Model;

namespace NetworkSim.View
{
    /// <summary>
    /// Панель, на которой рисуется сцена моделирования. Обладает двойной
    /// буферизацией (без мерцания) и центрирует логическую схему в окне.
    /// </summary>
    public class SimulationCanvas : Panel
    {
        private SimulationWorld _world;     // моделируемый мир

        /// <summary>Создаёт холст с двойной буферизацией.</summary>
        public SimulationCanvas()
        {
            DoubleBuffered = true;
            ResizeRedraw = true;
            BackColor = Color.FromArgb(250, 250, 251);
        }

        /// <summary>Задаёт моделируемый мир для отображения.</summary>
        public void SetWorld(SimulationWorld world)
        {
            _world = world;
            Invalidate();
        }

        /// <summary>Отрисовка сцены: качество, масштаб, центрирование.</summary>
        protected override void OnPaint(PaintEventArgs e)
        {
            base.OnPaint(e);
            if (_world == null)
            {
                return;
            }
            float scale = Math.Min(Width / (float)Model.Layout.CanvasWidth,
                                   Height / (float)Model.Layout.CanvasHeight);
            if (scale <= 0f)
            {
                return;
            }
            Graphics g = e.Graphics;
            g.SmoothingMode = SmoothingMode.AntiAlias;
            g.TextRenderingHint = TextRenderingHint.ClearTypeGridFit;
            g.TranslateTransform((Width - Model.Layout.CanvasWidth * scale) / 2f,
                                 (Height - Model.Layout.CanvasHeight * scale) / 2f);
            g.ScaleTransform(scale, scale);
            _world.Draw(g);
        }
    }
}
```

**Файл MainForm.cs**

```
using System;
using System.Windows.Forms;
using NetworkSim.Model;

namespace NetworkSim
{
    /// <summary>
    /// Главное окно приложения: панель настроек модели, сцена моделирования,
    /// журнал событий и строка состояния со счётчиками. Создаёт мир модели
    /// (SimulationWorld) и обрабатывает команды пользователя.
    /// </summary>
    public partial class MainForm : Form
    {
        private SimulationWorld _world;     // моделируемый мир
```

```
/// <summary>Создаёт главное окно и запускает модель в собранном виде.</summary>
public MainForm()
{
    InitializeComponent();
    InitSettingsControls();
    CreateWorld();
}

// Элемент выпадающего списка: число + готовый текст (без зависимости
// от языковых настроек, где разделитель может быть точкой).
private class ValueItem
{
    private readonly string _text;

    public double Value { get; }

    public ValueItem(double value, string text)
    {
        Value = value;
        _text = text;
    }

    public override string ToString()
    {
        return _text;
    }
}

// Заполняет выпадающие списки допустимыми значениями параметров.
private void InitSettingsControls()
{
    cmbDataError.Items.Add(new ValueItem(0.0, "0,0"));
    cmbDataError.Items.Add(new ValueItem(0.1, "0,1"));
    cmbDataError.Items.Add(new ValueItem(0.2, "0,2"));
    cmbDataError.Items.Add(new ValueItem(0.3, "0,3"));
    cmbDataError.SelectedIndex = 1;

    cmbOutage.Items.Add(new ValueItem(0.0, "0,0"));
    cmbOutage.Items.Add(new ValueItem(0.05, "0,05"));
    cmbOutage.Items.Add(new ValueItem(0.1, "0,1"));
    cmbOutage.Items.Add(new ValueItem(0.2, "0,2"));
    cmbOutage.SelectedIndex = 1;

    cmbProcTime.Items.Add(new ValueItem(1.0, "1"));
    cmbProcTime.Items.Add(new ValueItem(2.0, "2"));
    cmbProcTime.Items.Add(new ValueItem(3.0, "3"));
    cmbProcTime.SelectedIndex = 1;
}

// Создаёт новый мир и подключает его к окну (запуск и кнопка «Сброс»).
private void CreateWorld()
{
    if (_world != null)
    {
        _world.Clock.Tick -= OnClockTick;
        _world.Logger.MessageLogged -= OnLogMessage;
        _world.Clock.Stop();
    }
    _world = new SimulationWorld();
    _world.Client.AutoSend = chkAuto.Checked;
    _world.Clock.SpeedFactor = trbSpeed.Value / 2.0;
    canvas.SetWorld(_world);
    _world.Clock.Tick += OnClockTick;
    _world.Logger.MessageLogged += OnLogMessage;
    ApplySettingsFromUi();
    btnStartStop.Text = "Старт";
    lstLog.Items.Clear();
    _world.Logger.Log("Модель", "модель готова к работе, нажмите «Старт»");
```

```
            UpdateStatusLabel();
            canvas.Invalidate();
        }

        // Применяет выбранные настройки к модели. Недопустимое значение
        // приводит к исключению, которое перехватывается и показывается.
        private void ApplySettingsFromUi()
        {
            if (_world == null)
            {
                return;
            }
            try
            {
                double dataError = ((ValueItem)cmbDataError.SelectedItem).Value;
                double outage = ((ValueItem)cmbOutage.SelectedItem).Value;
                double processingTime = ((ValueItem)cmbProcTime.SelectedItem).Value;
                _world.ApplySettings(dataError, outage, processingTime);
            }
            catch (ArgumentOutOfRangeException ex)
            {
                MessageBox.Show(ex.Message, "Недопустимое значение параметра",
                    MessageBoxButtons.OK, MessageBoxIcon.Error);
            }
        }

        // Шаг модельного времени: обновить счётчики и перерисовать сцену.
        private void OnClockTick(object sender, TickEventArgs e)
        {
            UpdateStatusLabel();
            canvas.Invalidate();
        }

        // Новая запись журнала: добавить строку, ограничить длину и прокрутить.
        private void OnLogMessage(object sender, LogEventArgs e)
        {
            lstLog.Items.Add(e.Line);
            while (lstLog.Items.Count > 300)
            {
                lstLog.Items.RemoveAt(0);
            }
            lstLog.TopIndex = lstLog.Items.Count - 1;
        }

        // Обновляет строку состояния со счётчиками модели.
        private void UpdateStatusLabel()
        {
            lblCounters.Text = string.Format(
                "Время: {0:0.0} с      Отправлено: {1}      Доставлено: {2}      Возвращено ошибок:
{3}" +
                "      Ремонтов: {4}      Задач в полёте: {5}",
                _world.Clock.Elapsed, _world.SentCount, _world.DeliveredCount,
                _world.ErrorsReturned, _world.RepairsDone, _world.TasksActive);
        }

        // «Старт/Стоп»: запуск и остановка модельного времени.
        private void btnStartStop_Click(object sender, EventArgs e)
        {
            if (_world.Clock.IsRunning)
            {
                _world.Clock.Stop();
                btnStartStop.Text = "Старт";
            }
            else
            {
                _world.Clock.Start();
                btnStartStop.Text = "Стоп";
            }
            canvas.Invalidate();
        }
```

```
    // «Задача»: отправка новой задачи по команде пользователя.
    private void btnSendTask_Click(object sender, EventArgs e)
    {
        string reason;
        if (_world.TrySendTask(out reason))
        {
            lblHint.Text = "";
        }
        else
        {
            lblHint.Text = reason;
        }
        canvas.Invalidate();
    }

    // «Сброс»: полная пересборка модели в исходное состояние.
    private void btnReset_Click(object sender, EventArgs e)
    {
        CreateWorld();
        lblHint.Text = "";
    }

    // «Скрыть журнал»: освобождает место для сцены моделирования.
    private void btnLog_Click(object sender, EventArgs e)
    {
        lstLog.Visible = !lstLog.Visible;
        btnLog.Text = lstLog.Visible ? "Скрыть журнал" : "Показать журнал";
        canvas.Invalidate();
    }

    private void cmbDataError_SelectedIndexChanged(object sender, EventArgs e)
    {
        ApplySettingsFromUi();
    }

    private void cmbOutage_SelectedIndexChanged(object sender, EventArgs e)
    {
        ApplySettingsFromUi();
    }

    private void cmbProcTime_SelectedIndexChanged(object sender, EventArgs e)
    {
        ApplySettingsFromUi();
    }

    // Скорость: положение движка (1–8) делится на 2 — от 0,5 до 4.
    private void trbSpeed_ValueChanged(object sender, EventArgs e)
    {
        if (_world != null)
        {
            _world.Clock.SpeedFactor = trbSpeed.Value / 2.0;
        }
    }

    private void chkAuto_CheckedChanged(object sender, EventArgs e)
    {
        if (_world != null)
        {
            _world.Client.AutoSend = chkAuto.Checked;
        }
    }

    // При закрытии окна обязательно останавливаем модельное время.
    private void MainForm_FormClosing(object sender, FormClosingEventArgs e)
    {
        _world.Clock.Stop();
    }
}
```

```
}
```

# 5 РЕЗУЛЬТАТЫ ТЕСТИРОВАНИЯ

Тестирование выполнялось вручную по сценариям, охватывающим успешную обработку, оба вида сбоев, очередь, ограничения и элементы управления. Для каждого сценария задавались исходные данные, фиксировался ожидаемый и фактический результат. Результаты приведены в таблице 6.

Таблица 6 – Результаты тестирования программы

| № | Сценарий | Исходные данные | Ожидаемый результат | Фактический результат | Вердикт |
|---|---|---|---|---|---|
| 1 | Успешная обработка | Вероятности 0; 1 задача | Задача проходит цепочку, результат у клиента, пользователь радуется | Счётчики 1/1/0/0/0, результат получен (рисунок 4) | Пройден |
| 2 | Ошибка данных | Ошибка 0,3; отказ 0; 3 задачи | Ошибки возвращаются клиенту, выполняются повторные попытки | Возвращено 2–3 ошибки, попытки 2 и 3 (рисунок 12) | Пройден |
| 3 | Очередь пакетов | Вероятности 0; 5 задач; скорость 0,5× | Под узлом видны пакеты, ожидающие обработки | Под коммутатором 4 пакета в лотке (рисунок 5) | Пройден |
| 4 | Отказ базы данных | Отказ 0,2; 5 задач | Узел краснеет, администратор ремонтирует, обработка продолжается | Ремонт выполнен, очередь обработана (рисунок 13) | Пройден |
| 5 | Предел задач в полёте | Шесть нажатий «Задача» | Шестая задача не создаётся, выводится подсказка | Отправлено 5, подсказка о пределе (рисунок 14) | Пройден |
| 6 | Автоотправка | Флажок включён; 7 с | Задачи отправляются автоматически каждые 6 с | Пакет #7 отправлен в 6,0 с (рисунок 15) | Пройден |
| 7 | Изменение скорости | Движок 0,5×…4× | Модельное время ускоряется или замедляется | Время моделирования меняется пропорционально | Пройден |
| 8 | Остановка и продолжение | «Стоп» на 2 с, затем «Старт» | Модель замирает и продолжает с того же места | Время 3,8 с не изменилось за паузу, затем выросло | Пройден |
| 9 | Сброс на ходу | Пакеты в пути | Чистое начальное состояние без исключений | Сцена и журнал очищены, модель работает | Пройден |
| 10 | Скрытие журнала | Кнопка «Скрыть журнал» | Сцена занимает освободившееся место | Журнал скрыт, сцена расширена (рисунок 16) | Пройден |

На рисунке 12 показан возврат ошибки данных: клиентский сегмент перешёл в состояние «авария», клиент исправляет ошибку, к серверу уже летит повторная попытка.

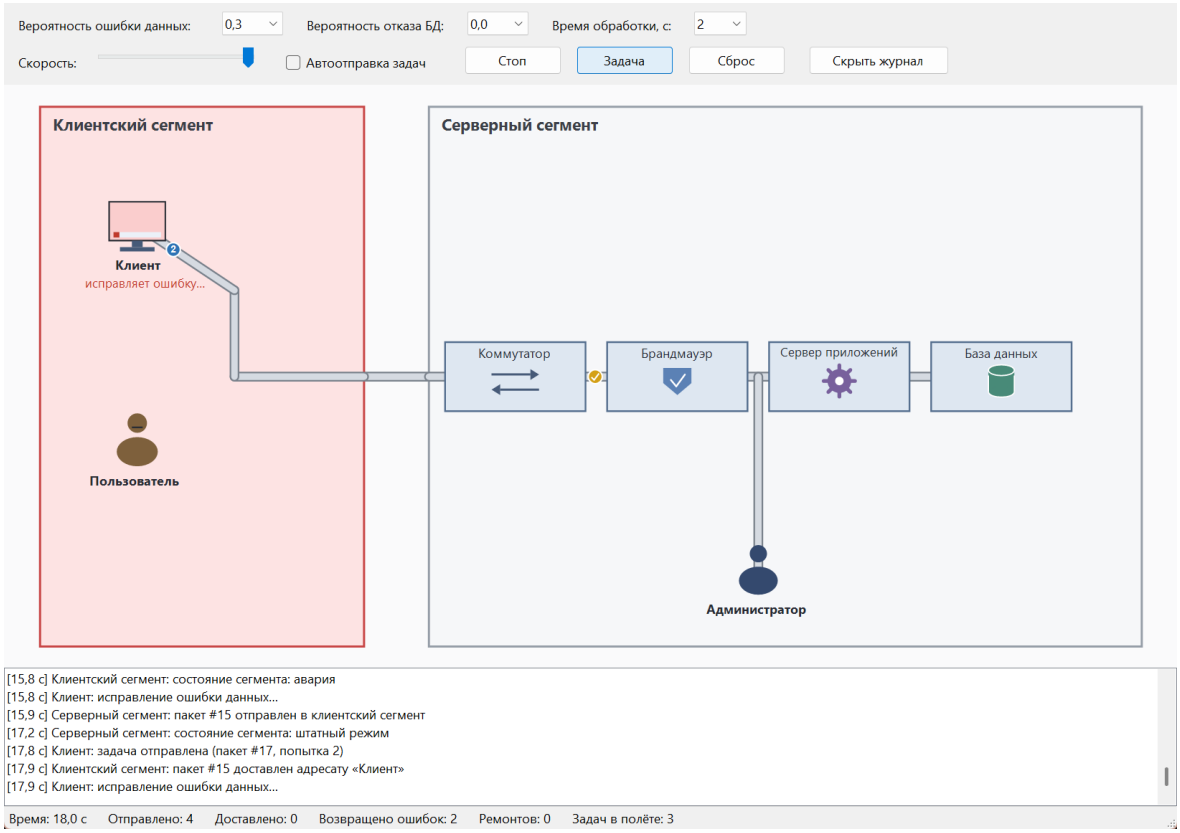



Рисунок 12 – Возврат ошибки данных и повторная попытка

На рисунке 13 показан отказ базы данных: узел красный, к нему ведёт пунктирная линия SSH, под узлом накапливается очередь, администратор выполняет ремонт.

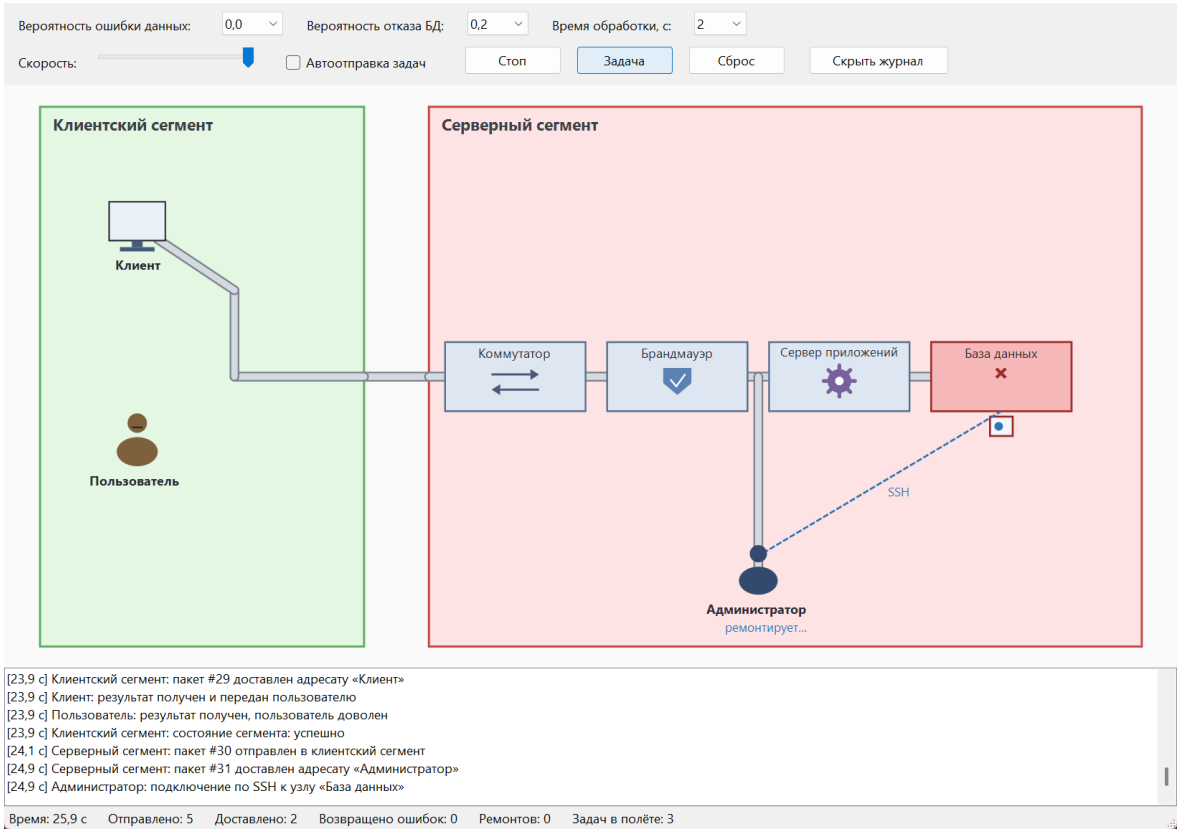



Рисунок 13 – Отказ базы данных и работа администратора

На рисунке 14 показано ограничение числа задач: после пяти отправленных задач шестая не создаётся, в строке состояния выводится подсказка.

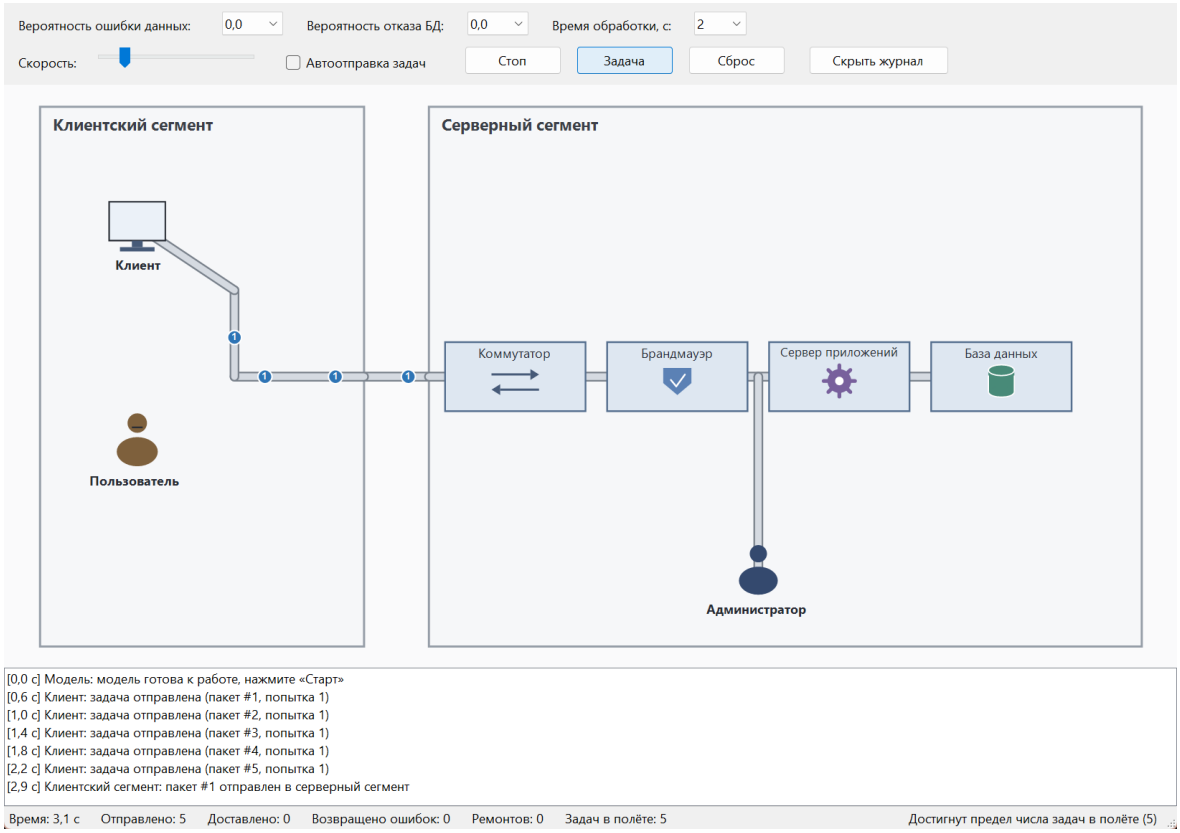



Рисунок 14 – Ограничение числа задач в полёте

На рисунке 15 показана автоматическая отправка: флажок включён, задача отправлена без участия пользователя.

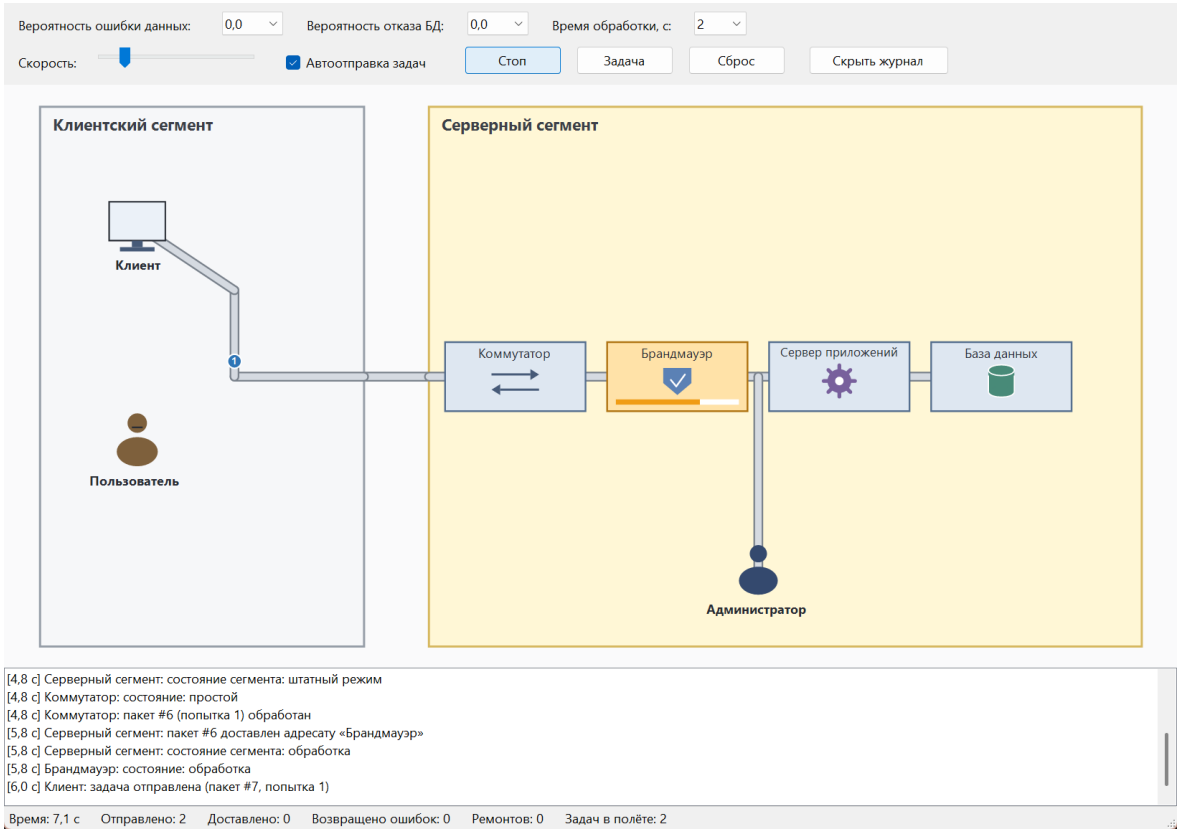



Рисунок 15 – Автоматическая отправка задач

На рисунке 16 показан режим без журнала: сцена занимает всё окно и масштабируется под новый размер.

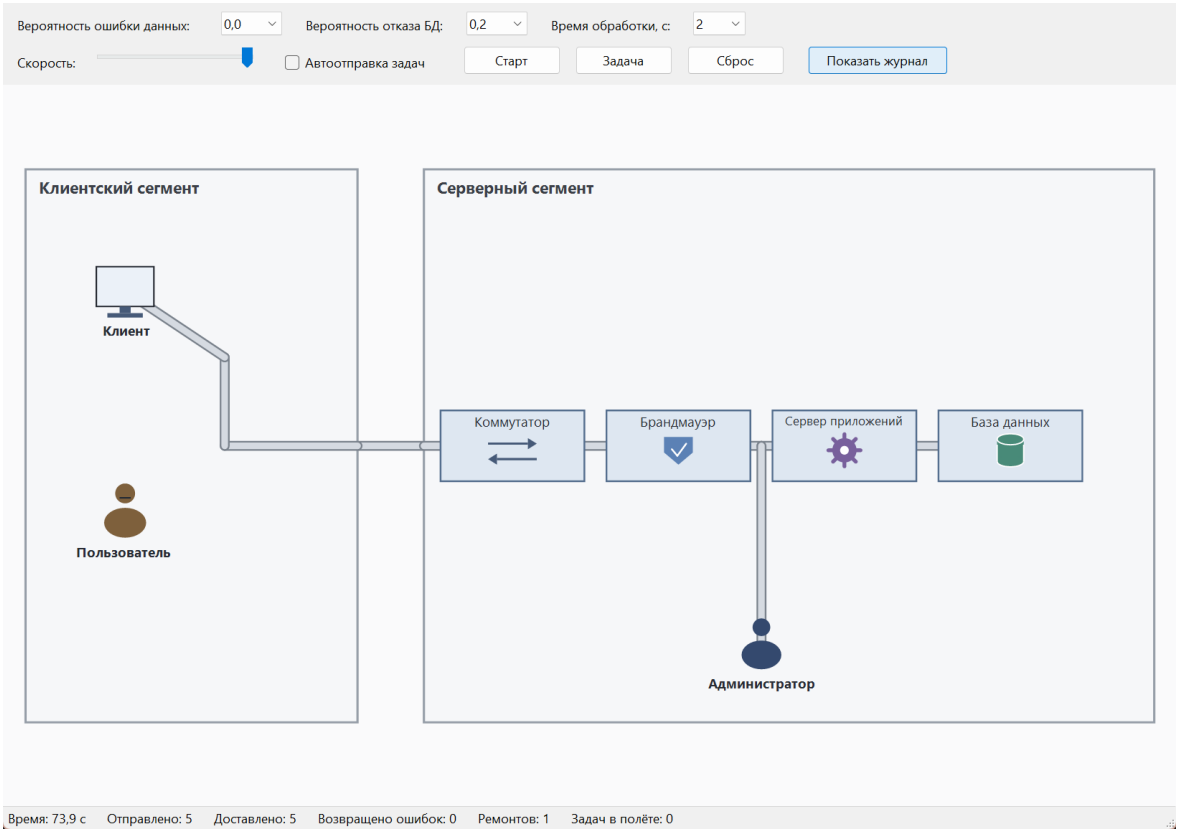



Рисунок 16 – Главное окно со скрытым журналом

Все сценарии выполнены без исключений и зависаний. Программа корректно реагирует на остановку моделирования, предел задач, сброс и закрытие окна.

# ВЫВОДЫ

В ходе работы разработано приложение «Модель работы вычислительной сети при выполнении задачи» на языке C# с использованием Windows Forms. Приложение моделирует прохождение задачи по цепочке из четырёх узлов, два вида сбоев, работу клиента и администратора, очередь пакетов и визуальное изменение состояний объектов.

Задание 1 выполнено: основной класс ProcessingNode содержит инкапсулированные данные, восемь свойств, два метода, конструктор и три события; для свойств реализована проверка значений. Созданы вспомогательные классы, необходимые для моделирования.

Задание 2 выполнено: созданы три класса-наследника узла и два класса-наследника сегмента, переопределены методы Evaluate, Draw и ResolveDestination. Взаимодействие объектов организовано через собственные делегаты и события: два события объявлены с явными методами add и remove, две подписки выполнены в другом классе через явное создание делегатов.

Проведено тестирование по десяти сценариям, включая успешную обработку, ошибку данных, отказ базы данных, очередь, предел задач, автоотправку, изменение скорости, паузу, сброс и скрытие журнала. Все сценарии прошли успешно.

Код разделён на модель и представление, снабжён комментариями на русском языке и может быть использован для дальнейшего развития: добавления новых типов узлов, сбоев и сценариев обработки.